408 计算机组成原理：指令系统（考研复习笔记）

指令系统是计算机组成原理中非常重要的一章，也是408中比较适合通过掌握固定题型来拿分的章节。

这一章的特点是：**概念多，但计算题的套路比较固定。** 尤其是指令格式、扩展操作码、寻址方式、有效地址计算、相对寻址，掌握方法后通常比较容易得分。

结合你目前以“保基础分、优先掌握常考题型”为目标的复习策略，我把笔记分成三个层次：

- 必会 高频考点，计算题必须掌握。
- 重点 常见选择题和综合题考点。
- 了解 以概念辨析为主。

一、指令系统的基本概念

1. 什么是指令?

指令（Instruction）：计算机执行某种操作的命令。

指令系统（Instruction Set）：一台计算机所能执行的全部机器指令的集合。

指令系统是软件和硬件之间的重要接口，属于计算机体系结构中对程序员可见的部分。

2. 指令的基本格式

一条机器指令通常由两部分组成：



基本指令格式示意图（各字段长度依具体指令系统而定）

字段	作用	例子
操作码 OP	指明指令的操作类型	ADD、SUB、LOAD
地址码 A	指明操作数、结果位置或寻址信息	寄存器号、内存地址

例如：

```asm
ADD R1, R2
```

表示：

$$R1 \leftarrow R1 + R2$$

即把寄存器 R1 和 R2 中的数据相加，结果保存到 R1。

注意：具体指令的源操作数与目的操作数顺序由指令集约定。

3. 指令字长

指令字长：一条机器指令所占的二进制位数。

常见三种：

类型	特点
定长指令字	所有指令长度相同
变长指令字	不同指令长度不同
混合形式	部分固定、部分可变

必须区分三个概念：

- 机器字长：CPU 一次能够处理的数据位数（通常与通用寄存器宽度有关）。
- 存储字长：一个存储单元所包含的二进制位数。
- 指令字长：一条指令的位数。

三者不一定相等。

例如：

- 机器字长：32 位
- 存储字长：8 位
- 指令字长：32 位

那么一条指令占：

$$\frac{32}{8} = 4 \text{ 个存储单元}$$

若按字节编址，就是 4 B。

二、指令的地址数 重点

按照地址码数量，可以分为：

- 零地址指令
- 一地址指令
- 二地址指令
- 三地址指令
- 四地址指令（较少考查）

1. 三地址指令

格式：

$$OP \quad A_1 \quad A_2 \quad A_3$$

例如：

```asm
ADD A, B, C
```

表示：

$$(A) + (B) \rightarrow C$$

两个源操作数分别来自 A、B，结果写入 C。

2. 二地址指令

格式：

$$OP \quad A_1 \quad A_2$$

典型含义：

$$(A_1)\ OP\ (A_2) \rightarrow A_1$$

例如：

```asm

```

```
ADD R1, R2
```

$$R1 \leftarrow R1 + R2$$

其中 R1 既是源操作数，也是目的操作数。

3. 一地址指令

格式：

$$OP \quad A$$

两种常见情况：

情况①：单操作数指令

```
asm
INC A
```

$$(A) + 1 \rightarrow A$$

情况②：隐含累加器 ACC

```
asm
ADD A
```

$$(ACC) + (A) \rightarrow ACC$$

4. 零地址指令

格式：

$$OP$$

常见于堆栈计算机。

例如：

```
asm
ADD
```

从栈顶弹出两个数，相加后把结果压回栈中。

也可能用于不需要显式操作数的指令，例如某些返回指令。

5. 考试常见结论

一般而言：

- 显式地址越多，指令长度可能越长。
- 显式地址越少，对隐含寄存器、堆栈等机制的依赖可能越大。
- 零地址指令不代表不需要操作数。
- 一地址指令不代表只能有一个操作数。

三、操作码与扩展操作码 必会

这是本章最值得优先拿下的计算题型之一。

1. 定长操作码

假设操作码有 n 位，则最多表示：

$$\boxed{2^n \text{ 种操作}}$$

例如：

操作码长 6 位：

$$2^6 = 64$$

最多可以表示 64 种操作。

反过来，如果需要表示 N 种操作，那么至少需要：

$$\boxed{n = \lceil \log_2 N \rceil}$$

其中 $\lceil \ \rceil$ 表示向上取整。

例如需要 100 种操作：

$$2^6 = 64 < 100$$

$$2^7 = 128 \geq 100$$

所以操作码至少需要 7 位。

2. 扩展操作码

扩展操作码：**利用不同长度的操作码表示不同类型的指令。**

一般来说：

- 地址数较多的指令，操作码较短。
- 地址数较少的指令，操作码可以较长。

目的：提高指令编码空间的利用率。

经典题型：求最多可以设计多少条指令

假设：

- 指令字长：16 位
- 每个地址字段：4 位
- 采用扩展操作码
- 已设计 15 条三地址指令
- 问最多还能设计多少条二地址指令?

第一步：求三地址指令的操作码长度。

三地址字段共：

$$3 \times 4 = 12 \text{ 位}$$

因此操作码：

$$16 - 12 = 4 \text{ 位}$$

4 位操作码最多表示：

$$2^4 = 16$$

种三地址指令。

第二步：计算剩余操作码。

已经使用 15 个。

剩余：

$$16 - 15 = 1$$

个操作码前缀可供扩展。

第三步：计算二地址指令的扩展位数。

二地址指令占用：

$$2 \times 4 = 8 \text{ 位}$$

因此二地址操作码长度：

$$16 - 8 = 8 \text{ 位}$$

比三地址操作码多：

$$8 - 4 = 4 \text{ 位}$$

第四步：计算最多可以设计多少条。

$$N = 1 \times 2^4$$

$$\boxed{N = 16}$$

答案：最多还能设计 16 条二地址指令。

可交互练习：扩展操作码计算

修改参数，观察二地址指令的最大数量

指令字长

每个地址字段长度 **4 位**

已使用的三地址指令数量 **15 条**

剩余操作码前缀
1

最多二地址指令
16

计算：(16 − 15) × 2^4 = 16

3. 扩展操作码通用公式

设：

- n：操作码初始长度
- m：每个地址字段长度
- k：已经使用的初始操作码数量

若从三地址扩展为二地址，且其余条件符合上述模型：

$$\boxed{N = (2^n - k) \times 2^m}$$

继续从二地址扩展到一地址，思路相同：

$$\boxed{\text{剩余前缀数} \times 2^{\text{新增操作码位数}}}$$

注意：扩展时必须保留足够的编码前缀，不能让短操作码与长操作码发生冲突。

考试口诀：

> 先算总数，再减已用；少一个地址，多一段操作码；剩余前缀乘扩展数量。

四、指令寻址与数据寻址 必会

这两个概念一定要区分。

类型	解决的问题
指令寻址	下一条指令在哪里?
数据寻址	当前指令的操作数在哪里?

1. 指令寻址

指令寻址主要分为：

- 顺序寻址
- 跳跃寻址

（1）顺序寻址

程序按照指令存储顺序执行。

一般情况下：

$$\boxed{PC \leftarrow PC + \text{指令长度}}$$

这里的指令长度必须换算为与 PC 相同的地址单位。

例如：

- 按字节编址
- 每条指令长度 4 B
- 当前 PC = 1000

取完指令后：

$$PC = 1004$$

注意：有些教材用 $PC + 1$，这是因为默认一条指令对应一个编址单位。

（2）跳跃寻址

执行转移指令时，会改变 PC。

例如：

```asm
JMP 2000
```

执行后：

$$PC \leftarrow 2000$$

此时下一条指令不一定是原来顺序存放的下一条指令。

五、数据寻址方式 必会

这是408的高频重点。

先掌握三个符号：

符号	含义
A	指令地址字段中的形式地址
EA	有效地址（操作数所在位置的地址）
M[A]	内存地址 A 中存放的内容

例如：

假设：

$$M[100] = 200$$

表示内存地址 100 中存储的数值为 200。

注意：以下公式针对典型指令格式，具体机器可能存在区别。

1. 立即寻址

操作数直接存放在指令中。

例如：

```asm
MOV R1, #100
```

表示：

$$R1 \leftarrow 100$$

这里的 100 就是操作数本身。

特点：

- 不需要为了取得该操作数而再次访问内存。
- 速度快。
- 操作数大小受指令字段长度限制。

口诀：立即寻址，给的就是数。

2. 直接寻址

指令地址字段 A 直接给出操作数的有效地址。

$$\boxed{EA = A}$$

例如：

```asm
```

```
LOAD R1, [100]
```

假设：

$$M[100] = 500$$

则：

$$R1 \leftarrow 500$$

理解：

```
纯文本
形式地址 A = 100
有效地址 EA = 100
操作数 M[100] = 500
```

特点：

- 计算简单。
- 取操作数通常需要访问一次内存。
- 可寻址范围受地址字段长度限制。

口诀：直接寻址，给的是地址。

3. 间接寻址

指令给出的 A 不是操作数地址，而是存放操作数地址的位置。

$$\boxed{EA = M[A]}$$

例如：

$$A = 100$$

$$M[100] = 200$$

$$M[200] = 500$$

则：

$$EA = M[100] = 200$$

操作数为：

$$M[EA] = M[200] = 500$$

一次间接寻址过程

指令地址字段：A = 100

↓

M[100] = 200（有效地址）

↓

M[200] = 500（操作数）

特点：

- 可扩大可寻址范围。
- 便于实现指针。
- 为取得操作数，通常需要访问内存两次。

口诀：间接寻址，先找地址，再找数据。

4. 寄存器寻址

操作数位于寄存器中。

例如：

```asm
ADD R1, R2
```

操作数来自 R2。

特点：

- 访问速度快。
- 不需要额外访问主存获取该寄存器操作数。
- 寄存器数量有限。

5. 寄存器间接寻址

寄存器中保存的是操作数的内存地址。

$$\boxed{EA = (R)}$$

这里的 (R) 表示寄存器 R 中的内容。

假设：

$$(R1) = 100$$

$$M[100] = 500$$

则：

$$EA = 100$$

操作数：

$$M[100] = 500$$

对比：

寻址方式	寄存器里放什么？
寄存器寻址	操作数
寄存器间接寻址	操作数的地址

注意：寄存器间接寻址通常只需一次主存访问来取得操作数。

6. 基址寻址

$$\boxed{EA = (BR) + A}$$

其中：

- BR：基址寄存器
- A：形式地址（位移量）

例如：

$$(BR) = 1000$$

$$A = 200$$

所以：

$$EA = 1000 + 200 = 1200$$

作用：

- 便于程序重定位。
- 便于实现存储保护。
- 可扩大寻址范围。

典型特点：

基址寄存器的值通常由操作系统或相关机制设定，程序员一般通过改变位移量访问不同地址。

7. 变址寻址

$$\boxed{EA = (IX) + A}$$

其中 IX 为变址寄存器。

例如：

$$(IX) = 20, \quad A = 1000$$

则：

$$EA = 1020$$

作用：

- 数组访问
- 循环操作
- 顺序遍历

典型特点：

形式地址通常固定，变址寄存器的值可以随着循环改变。

基址与变址对比

比较	基址寻址	变址寻址
公式	$EA = (BR) + A$	$EA = (IX) + A$
主要作用	程序重定位	数组遍历
典型变化部分	形式地址 A	变址寄存器 IX
典型控制者	系统相关机制	用户程序

考试注意：两者公式相似，但主要应用目的不同。

8. 相对寻址

这是非常重要的计算题型。

$$\boxed{EA = (PC) + A}$$

其中：

- PC：程序计数器
- A：相对位移量，通常为有符号数

特别注意：

相对寻址计算时，通常使用取出当前指令后、已经更新的 PC。

例题

某计算机按字节编址：

- 当前指令地址：1000
- 当前指令长度：4 B
- 相对位移量：20

求目标地址。

第一步：更新 PC。

$$PC = 1000 + 4 = 1004$$

第二步：计算目标地址。

$$EA = 1004 + 20$$

$$\boxed{EA = 1024}$$

如果位移量为 -20：

$$EA = 1004 - 20 = 984$$

相对寻址容易出现的陷阱

1. 忘记 PC 已经更新。
2. 将负位移量当成正数。
3. 忘记符号扩展。
4. 混淆按字节编址和按字编址。
5. 忽略某些指令集对位移量的缩放规则。

考试口诀：先更新 PC，再加位移。

9. 隐含寻址

指令中不显式给出操作数地址，而是由指令规定隐含位置。

例如某些使用累加器 ACC 的指令：

```asm
CMA
```

可能表示对累加器内容取反。

10. 堆栈寻址

操作数默认位于栈顶。

常见寄存器：

$$SP$$

即栈指针。

典型操作：

- PUSH：入栈
- POP：出栈

注意：

- 栈可以向高地址增长，也可以向低地址增长。
- SP 指向栈顶元素还是下一个空位置，由机器约定。

六、各种寻址方式总结表

这张表建议重点记忆。

寻址方式	有效地址 / 操作数	典型特点
立即寻址	操作数 = A	不需要额外取操作数
直接寻址	$EA = A$	地址字段直接给地址
间接寻址	$EA = M[A]$	先取地址，再取数据
寄存器寻址	操作数 = (R)	寄存器直接存数据
寄存器间接寻址	$EA = (R)$	寄存器存内存地址
基址寻址	$EA = (BR) + A$	程序重定位
变址寻址	$EA = (IX) + A$	数组访问
相对寻址	$EA = (PC) + A$	转移指令
隐含寻址	由指令隐含指定	无需显式地址字段
堆栈寻址	由 SP 等确定	栈操作

高频考点：访存次数

假设指令已经取出，只计算获取一个源操作数所需的主存访问次数，不考虑 Cache 和分页地址转换。

寻址方式	取操作数访存次数
立即寻址	0
寄存器寻址	0
直接寻址	1
一次间接寻址	2
寄存器间接寻址	1
基址寻址	1
变址寻址	1
相对寻址	1（若目标操作数在内存）

注意：相对寻址常用于转移指令，若只是修改 PC，不需要像读取内存数据那样再访问目标地址获取操作数。

七、相对寻址与补码计算 必会

相对位移量经常采用补码表示。

例题

已知：

- PC = 2000H（已经更新）
- 位移量为 8 位补码：FCH
- 位移量以字节为单位

求目标地址。

第一步：把 FCH 转为二进制。

$$FC_H = 11111100_2$$

最高位为 1，说明是负数。

第二步：求真值。

按位取反：

$$00000011$$

加 1：

$$00000100 = 4$$

因此：

$$FC_H = -4$$

第三步：计算目标地址。

$$EA = 2000_H - 4$$

$$\boxed{EA = 1FFC_H}$$

快速判断补码负数

如果是 8 位补码，数值大于等于 80H，则对应有符号负数。

例如：

补码	真值
7FH	127
FFH	-1
FEH	-2
FCH	-4
F0H	-16
80H	-128

快捷方法：

$$\boxed{\text{负数真值} = \text{无符号值} - 2^n}$$

例如：

$$FC_H = 252$$

$$252 - 256 = -4$$

八、指令格式计算题 必会

这类题也非常适合通过固定步骤解决。

1. 根据指令长度计算地址字段

假设：

- 指令长度 32 位
- 操作码 8 位
- 两地址指令
- 两个地址字段等长

求每个地址字段长度。

$$32 = 8 + 2A$$

所以：

$$2A = 24$$

$$\boxed{A = 12 \text{ 位}}$$

2. 根据寄存器数量计算寄存器编号位数

假设计算机有 32 个通用寄存器。

则寄存器编号至少需要：

$$\lceil \log_2 32 \rceil = 5$$

即：

$$\boxed{5 \text{ 位}}$$

例如：

```
纯文本
00000 → R0
00001 → R1
...
11111 → R31
```

3. 根据地址字段计算寻址范围

假设地址字段长 12 位，直接寻址。

则可表示的地址数量为：

$$2^{12} = 4096$$

如果按字节编址：

$$4096B = 4KiB$$

如果按字编址，每个字长 4 B：

$$4096 \times 4B = 16KiB$$

注意区分：地址数量和实际字节容量。

九、指令系统设计：CISC 与 RISC 重点

408选择题常见考点。

1. CISC

复杂指令集计算机（Complex Instruction Set Computer）。

典型代表：x86 指令集。

特点：

- 指令种类较多。
- 指令格式较复杂。
- 指令长度可以不固定。
- 寻址方式较丰富。
- 一条指令可能完成较复杂的操作。

2. RISC

精简指令集计算机（Reduced Instruction Set Computer）。

典型代表：RISC-V、ARM（经典 RISC 风格）。

特点：

- 指令设计倾向于简单、规整。
- 寻址方式相对较少。
- 通用寄存器数量通常较多。
- 通常采用 Load/Store 架构。
- 便于流水线设计和硬件实现。

CISC 与 RISC 对比

对比项目	CISC	RISC
指令种类	较多	相对较少
指令复杂度	较高	相对较低
指令长度	常见变长	常见定长或规整编码
寻址方式	较丰富	相对较少
通用寄存器	传统设计中相对较少	通常较多
存储器操作	可支持复杂内存操作	典型 Load/Store
流水线实现	解码可能较复杂	通常较容易

注意：这是典型设计倾向，不是绝对规则。例如 RISC-V 支持压缩指令扩展，现代处理器的微体系结构也可能包含复杂的译码与微操作机制。

高频选择题结论

1. RISC 并非指令越少越好。
2. RISC 一般使用较多的通用寄存器。
3. Load/Store 架构通常只有访存指令直接访问主存数据。
4. RISC 的规整指令格式有利于流水线实现。

5. CISC 不代表一定比 RISC 慢，性能取决于具体实现。

十、指令系统中的常见指令类型 重点

类型	典型指令	作用
数据传送	MOV、LOAD、STORE	数据复制或读写
算术运算	ADD、SUB、MUL、DIV	数值运算
逻辑运算	AND、OR、XOR、NOT	位运算
移位操作	SHL、SHR	数据移位
比较指令	CMP	比较操作数
条件转移	BEQ、BNE	满足条件时跳转
无条件转移	JMP	直接跳转
子程序调用	CALL	调用子程序
子程序返回	RET	返回调用位置
输入输出	IN、OUT	外设数据交换
系统控制	TRAP、特权指令等	系统服务和控制

注意：不同 ISA 中指令名称和具体语义不一定相同。

十一、程序状态字 PSW 与条件码 重点

指令执行后可能改变状态标志。

常见标志位：

标志	名称	含义
ZF	零标志	结果是否为 0
SF	符号标志	结果最高位是否为 1
CF	进位标志	无符号运算中的进位或借位情况
OF	溢出标志	有符号运算是否溢出

注意：具体标志更新规则取决于 ISA。

1. 进位 CF 与溢出 OF

这是经常混淆的概念。

- CF：主要用于判断无符号数运算的进位或借位。
- OF：用于判断有符号数运算是否溢出。

例题

8 位补码：

$$127 + 1$$

二进制：

$$01111111 + 00000001$$

结果：

$$10000000$$

若按 8 位有符号补码解释：

$$10000000 = -128$$

因此：

$$\boxed{OF = 1}$$

因为数学结果 128 超出了 8 位补码范围：

$$[-128, 127]$$

而最高位之外没有进位：

$$CF = 0$$

结论：有符号溢出和无符号进位是两件不同的事。

十二、408常考题型与固定解题步骤

这是你目前最应该重点记住的部分。

题型1：计算操作码至少需要多少位

识别关键词：

- 最多表示多少种指令?
- 至少需要多少位操作码?

直接套：

$$\boxed{n = \lceil \log_2 N \rceil}$$

例如 70 种操作：

$$2^6 = 64 < 70$$

$$2^7 = 128 \geq 70$$

答案：

$$\boxed{7\text{ 位}}$$

题型2：扩展操作码

识别关键词：

- 定长指令字
- 三地址、二地址、一地址
- 扩展操作码
- 最多还能设计多少条

固定步骤：

1. 算原操作码长度。
2. 算原操作码总数。
3. 减去已经使用的操作码数。
4. 算新指令格式增加的操作码位数。
5. 剩余数量乘以扩展倍数。

公式：

$$\boxed{N_{\text{新}} = N_{\text{剩余前缀}} \times 2^{\text{新增位数}}}$$

题型3：求有效地址 EA

识别关键词：

- 立即寻址
- 直接寻址
- 间接寻址
- 基址寻址
- 变址寻址
- 相对寻址

直接看公式：

$$\begin{aligned}
\text{直接：} &\quad EA = A \\
\text{间接：} &\quad EA = M[A] \\
\text{寄存器间接：} &\quad EA = (R) \\
\text{基址：} &\quad EA = (BR) + A \\
\text{变址：} &\quad EA = (IX) + A \\
\text{相对：} &\quad EA = (PC) + A
\end{aligned}$$

注意立即寻址是直接给操作数，并不需要计算内存有效地址。

题型4：相对转移地址

识别关键词：

- PC
- 位移量
- 补码
- 转移目标地址

固定步骤：

1. 判断题目给的是当前指令地址还是更新后的 PC。
2. 若需要，先计算取指后的 PC。
3. 将位移量补码转成有符号真值。
4. 按指令集规则处理位移量单位。
5. 计算目标地址。

一般公式：

$$\boxed{\text{目标地址} = \text{更新后PC} + \text{位移量}}$$

题型5：判断访存次数

识别关键词：

- 访问主存几次?
- 获取操作数需要几次访存?

先判断是否包含取指。

如果不包含取指：

- 立即寻址：0 次
- 寄存器寻址：0 次
- 直接寻址：1 次
- 寄存器间接寻址：1 次
- 一次存储器间接寻址：2 次

如果包含取指，并且指令本身只需要一次主存读出，则还要再加 1 次取指访存。

如果一条指令跨多个存储字，取指可能涉及多次存储器读取，不能机械地只加一次。

题型6：根据指令格式计算字段长度

识别关键词：

- 指令字长
- 操作码位数
- 寄存器个数
- 地址字段
- 寻址方式字段

固定方法：

$$\boxed{\text{指令总长度} = \text{各字段位数之和}}$$

例如：

- 指令长度 32 位
- 操作码 8 位
- 两个寄存器字段各 5 位
- 寻址方式字段 2 位

剩余位数：

$$32 - 8 - 5 - 5 - 2 = 12$$

因此还可容纳 12 位其他字段。

十三、408指令系统易错点总结

易错点	正确理解
指令字长等于机器字长	不一定
零地址指令没有操作数	可能隐含操作数
立即寻址需要访存获取操作数	不需要额外访存
间接寻址一次就能取到操作数	一次存储器间接寻址通常需两次访存
基址和变址作用完全相同	基址偏重重定位，变址偏重数组
相对寻址直接用当前指令首地址	通常使用更新后的 PC
位移量补码最高位为 1 就加上其无符号值	需要按有符号数解释
RISC 所有指令都必须等长	不一定
CF 和 OF 是同一个标志	CF 对应无符号进位/借位，OF 对应有符号溢出
操作码扩展时可以使用全部剩余二进制串	必须满足编码无歧义，避免前缀冲突

十四、考前速背版

如果考试前没时间了，这部分可以直接背下来。

指令系统必背 10 条

1. 指令 = 操作码 + 地址码。
2. n 位定长操作码最多表示 2^n 种操作。
3. 扩展操作码：剩余前缀数 × $2^{\text{新增位数}}$。
4. 直接寻址：$EA = A$。

5. 间接寻址：$EA = M[A]$。
6. 寄存器间接寻址：$EA = (R)$。
7. 基址寻址：$EA = (BR) + A$，主要用于重定位。
8. 变址寻址：$EA = (IX) + A$，主要用于数组。
9. 相对寻址：$EA = (PC) + A$，通常先更新 PC。
10. RISC：指令规整、寄存器较多、典型 Load/Store 架构。

十五、自测：6道基础题

基础巩固练习 0 / 6 已作答

1. 操作码长 7 位，最多能表示多少种操作?

A. 64

B. 128

C. 256

D. 512

2. 已知 M[100]=300、M[300]=500，采用一次间接寻址且 A=100，有效地址是多少?

A. 100

B. 300

C. 500

D. 800

3. 基址寻址最典型的作用是什么?

A. 数组遍历

B. 程序重定位

C. 减少寄存器

D. 算术移位

4. 相对寻址中，当前指令地址1000，长度4B，位移量-8B，目标地址为多少?

A. 992

B. 996

C. 1004

D. 1012

5. 下列哪种寻址方式通常需要两次数据主存访问（不计取指）?

A. 立即寻址

B. 直接寻址

C. 一次存储器间接寻址

D. 寄存器寻址

6. 8 位补码 11111100 对应的十进制真值是?

A. 252

B. -2

C. -4

D. -128

最后：针对你目前的408复习策略

这一章我建议你按照下面的顺序拿分：

优先级	内容	掌握要求
第一梯队	寻址方式与 EA 计算	看到题立即列公式
第一梯队	相对寻址与补码	熟练计算转移地址
第一梯队	扩展操作码	记住固定解题步骤
第二梯队	指令格式与字段位数	熟悉位数分配
第二梯队	CISC 与 RISC	熟记对比
第二梯队	访存次数与地址数	能做常规选择题
第三梯队	各类指令的细节	了解常见功能

我的建议是：这一章先别在概念上花太长时间。重点把“扩展操作码、寻址方式、相对转移、指令格式”四类题练熟。

它们的题型相对固定，非常符合你现在希望通过“记住步骤 → 识别题型 → 套用公式 → 稳定拿分”的复习方式。

408 计算机组成原理：中央处理器（CPU）复习笔记

这一章是408计算机组成原理的重点，内容主要包括：

1. CPU的功能与基本结构
2. 指令执行过程
3. 数据通路
4. 控制器（硬布线控制器、微程序控制器）
5. 异常和中断机制
6. 指令流水线
7. 多处理器基本概念

结合你现在的复习策略，这一章建议采取 **“计算题优先、固定步骤优先、基础概念必须掌握”** 的方式。

尤其是：

- 必会 指令执行过程、数据通路、流水线性能计算
- 必会 流水线数据冒险、控制冒险、时钟周期
- 重点 硬布线控制器与微程序控制器
- 重点 中断和异常处理
- 了解 多处理器、SIMD、MIMD等概念

一、CPU的功能与基本结构

1. CPU的主要功能

CPU主要由**运算器和控制器**组成。

部件	主要功能
运算器	执行算术运算和逻辑运算
控制器	负责取指令、分析指令、发出控制信号
寄存器组	暂存数据、地址和CPU状态
内部数据通路	在CPU内部传送数据

CPU主要完成：

- 指令控制：控制程序的执行顺序。
- 操作控制：产生控制信号，完成指令规定的操作。
- 时间控制：协调各操作发生的时序。
- 数据加工：完成算术和逻辑运算。
- 异常与中断处理：响应并处理异常事件。

2. CPU中的重要寄存器 必会

这张表非常重要，建议直接背下来。

寄存器	中文名称	作用
PC	程序计数器	存放下一条待取指令的地址
IR	指令寄存器	保存当前正在执行的指令
MAR	存储器地址寄存器	保存要访问的主存地址
MDR	存储器数据寄存器	暂存从主存读出或准备写入主存的数据
ACC	累加器	保存运算操作数和结果
PSW	程序状态字	保存条件码、状态及控制信息
GPR	通用寄存器	保存操作数、地址和计算结果

重点：哪些寄存器对程序员可见？

通常可见	通常不可见
通用寄存器 GPR	MAR
程序计数器 PC	MDR
程序状态字 PSW	IR
部分体系结构中的累加器 ACC	内部暂存寄存器

注意：

- PC、PSW一般属于对程序员可见的体系结构状态，但普通用户程序不一定能任意修改其中全部内容。
- MAR、MDR、IR通常属于CPU内部实现细节。
- 不同指令集架构中的具体规定可能不同。

高频易错点

PC存的是地址，IR存的是指令。

MAR存的是地址，MDR存的是数据。

例如：

$$PC = 1000$$

表示下一条待取指令的地址为1000。

取指后：

$$IR \leftarrow M[1000]$$

即IR保存取出的机器指令。

二、指令执行过程 必会

CPU执行指令的基本过程：



经典五阶段模型。它常用于讲解RISC流水线，并不意味着所有指令或CPU都严格经历这五个阶段。

1. 取指周期

典型取指操作：

$$MAR \leftarrow PC$$

$$MDR \leftarrow M[MAR]$$

$$IR \leftarrow MDR$$

$$PC \leftarrow PC + \text{指令长度}$$

解释：

1. PC提供指令地址。
2. MAR保存访存地址。
3. 主存把指令送入MDR。
4. MDR中的指令进入IR。
5. PC更新，准备顺序取下一条指令。

注意：以上是逻辑操作顺序。实际微操作安排可以因数据通路不同而改变，PC更新也可能与访存并行进行。

记忆口诀

PC找地址，MAR去访问；MDR收数据，IR存指令；PC再更新。

2. 间址周期

如果采用存储器间接寻址，需要额外获取操作数的有效地址。

例如：

$$EA = M[A]$$

微操作可以表示为：

$$MAR \leftarrow IR(A)$$

$$MDR \leftarrow M[MAR]$$

$$EA \leftarrow MDR$$

其中 $IR(A)$ 表示指令寄存器中的地址字段。

注意：间址周期是传统指令周期分析中的概念，不是所有指令都存在。

3. 执行周期

根据指令种类执行不同操作。

例如：

```asm
ADD R1, R2
```

$$R1 \leftarrow R1 + R2$$

对于访存指令：

```asm
LOAD R1, [R2]
```

假设R2保存地址，则：

$$MAR \leftarrow R2$$

$$MDR \leftarrow M[MAR]$$

$$R1 \leftarrow MDR$$

4. 中断周期

如果指令执行结束后需要响应中断，CPU通常执行：

1. 保存必要的程序现场。
2. 获取中断服务程序入口地址。
3. 转入中断服务程序。
4. 中断处理完成后恢复现场。
5. 返回原程序继续执行。

三、指令周期、机器周期、时钟周期 必会

这三个概念容易混淆。

1. 时钟周期

CPU时钟信号的一个周期。

$$\boxed{T = \frac{1}{f}}$$

其中：

- T：时钟周期
- f：时钟频率

例如：

$$f = 2GHz$$

则：

$$T = \frac{1}{2 \times 10^9}$$

$$\boxed{T = 0.5ns}$$

2. 机器周期

传统概念中，一个机器周期通常对应CPU完成一个基本操作阶段所需的时间，例如一次存储器访问。

一个机器周期可以包含多个时钟周期。

注意：现代CPU未必采用固定长度的机器周期。

3. 指令周期

CPU完成一条指令所需的时间。

典型关系：

$$\boxed{\text{指令周期包含一个或多个机器周期}}$$

传统多周期CPU中：

$$\text{机器周期包含一个或多个时钟周期}$$

高频结论

- 不同指令的指令周期可能不同。
- 取指是执行指令的必要环节。
- 间址周期不是所有指令都有。
- 中断周期不是每条指令都发生。
- 单周期CPU通常一条指令在一个时钟周期内完成。

四、数据通路 必会

数据通路是指CPU内部数据经过的路径及其相关部件。

主要包括：

- 寄存器
- ALU
- 多路选择器 MUX
- 总线
- 各种连接线路

控制器负责发出控制信号，使数据按照规定路径流动。

1. CPU内部单总线结构

多个寄存器和ALU通过一条内部总线交换数据。

特点：

- 结构简单。
- 硬件成本相对较低。
- 同一时刻通常只能有一个部件向总线输出数据。
- 数据传送可能需要分多个时钟节拍完成。

重点：为什么单总线通常需要暂存寄存器?

例如：

$$R1 \leftarrow R2 + R3$$

如果R2和R3都通过同一条总线送到ALU，那么它们通常不能同时在总线上传输。

所以需要暂存寄存器。

假设CPU具有暂存寄存器Y和Z：

$$Y \leftarrow R2$$

$$Z \leftarrow Y + R3$$

$$R1 \leftarrow Z$$

即：

1. 先把R2送入Y。
2. 再将Y与R3相加，结果保存到Z。
3. 最后将Z送入R1。

2. 单总线数据通路经典微操作

假设指令为：

```asm
ADD R1, R2
```

含义：

$$R1 \leftarrow R1 + R2$$

采用单总线结构，并设置Y、Z暂存寄存器。

时钟节拍	微操作
T1	$Y \leftarrow R1$
T2	$Z \leftarrow Y + R2$
T3	$R1 \leftarrow Z$

对应控制信号可以表示为：

节拍	控制信号示例
T1	R1out、Yin
T2	R2out、ALUadd、Zin
T3	Zout、R1in

其中：

- `R1out`： R1向总线输出。
- `Yin`： Y从总线接收数据。
- `ALUadd`： ALU执行加法。
- `Zin`： Z接收ALU输出。

具体控制信号名称以题目给定的数据通路为准。

3. 单总线取指微操作

对于某种带Y、Z暂存寄存器的数据通路，取指可能分为：

节拍	微操作
T1	$MAR \leftarrow PC$，同时计算 $PC+$ 指令长度
T2	$MDR \leftarrow M[MAR]$
T3	$IR \leftarrow MDR$

PC更新的具体节拍取决于题目中的硬件连接。

必须掌握的解题规则

单总线题目不要只背微操作顺序，一定要看数据通路图。

判断每个节拍能否同时执行，主要看：

1. 是否存在总线争用。
2. ALU输入是否已经准备好。
3. 寄存器之间是否有专用通路。
4. 是否涉及存储器读写等待。
5. 某个数据是否需要先暂存。

五、控制器 重点

控制器主要负责产生控制信号，协调CPU各部件工作。

常见两种实现方式：

- 硬布线控制器
- 微程序控制器

1. 硬布线控制器

通过组合逻辑电路、时序电路等直接产生控制信号。

典型输入信息：

- 指令操作码
- 时序信号
- 状态标志
- 外部控制信号

例如某个控制信号：

$$C = f(OP, T, PSW, \ldots)$$

其中：

- OP：操作码
- T：时序状态
- PSW：状态信息

特点

优点：

- 控制速度通常较快。
- 不需要读取控制存储器中的微指令。

缺点：

- 逻辑设计可能复杂。
- 修改和扩展指令控制逻辑较困难。

2. 微程序控制器

通过执行微程序来产生控制信号。

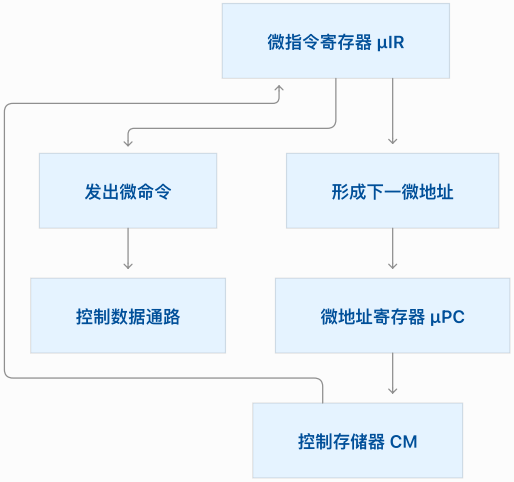
核心思想：

一条机器指令的执行过程，可以由一段微指令序列控制。

重要概念

名称	含义
微命令	控制某个具体微操作的控制信号
微操作	CPU内部执行的基本操作
微指令	规定一组微命令及后续控制信息
微程序	一系列微指令组成的序列
控制存储器 CM	保存微程序
微地址寄存器 μPC	保存待执行微指令的地址
微指令寄存器 μIR	保存当前微指令

微程序控制器的基本过程：



3. 微指令格式

一条微指令通常包含：

操作控制字段	顺序控制字段
控制执行哪些微操作	决定下一条微指令

4. 水平型与垂直型微指令

对比	水平型微指令	垂直型微指令
编码方式	直接或较少编码	较多编码
微指令长度	通常较长	通常较短
并行控制能力	较强	相对较弱
译码复杂度	较低	较高
微程序长度	通常较短	通常较长

高频结论

- 水平型微指令：**字长较长，并行性较好。**
- 垂直型微指令：**字长较短，编码较紧凑。**

六、微指令编码计算 必会

这类题具有固定解题步骤，建议重点练习。

1. 直接编码法

假设共有 n 个彼此独立的微命令。

每个微命令用一位表示：

- 1：发出该微命令
- 0：不发出该微命令

则操作控制字段长度：

$$\boxed{L = n}$$

例如有40个微命令：

$$L = 40 \text{ 位}$$

2. 字段直接编码法

将微命令分为若干组。

原则：

- 互斥的微命令可以放在同一字段。
- 可以并行执行的微命令通常应放在不同字段。

假设某字段有 m 个互斥微命令。

若需要保留一个编码表示“不发出任何微命令”，字段位数为：

$$\boxed{n = \lceil \log_2(m+1) \rceil}$$

例题

某微指令共有三组互斥微命令：

- 第一组：5个
- 第二组：7个
- 第三组：3个

每组都需要一种不操作状态。

求操作控制字段最少需要多少位。

第一组：

$$\lceil \log_2(5+1) \rceil = 3$$

第二组：

$$\lceil \log_2(7+1) \rceil = 3$$

第三组：

$$\lceil \log_2(3+1) \rceil = 2$$

总长度：

$$L = 3 + 3 + 2$$

$$\boxed{L = 8 \text{ 位}}$$

固定步骤

1. 看每组有几个互斥微命令。
2. 判断是否需要“不操作”编码。
3. 每组计算编码位数。
4. 将各组长度相加。
5. 如果题目还要求完整微指令长度，再加上地址字段等其他字段。

3. 后继微地址字段长度

假设控制存储器中有256条微指令。

要能够直接表示每条微指令的地址，至少需要：

$$\log_2 256 = 8$$

因此：

$$\boxed{8 \text{ 位}}$$

一般公式：

$$\boxed{n = \lceil \log_2 N \rceil}$$

其中N为需要编码的微地址数量。

若微指令同时包含操作控制字段和下地址字段，则：

$$\boxed{L = L_{\text{操作控制}} + L_{\text{下地址}} + \text{其他字段长度}}$$

注意：有些控制器通过增量计数器、转移条件或映射逻辑形成下一微地址，不一定在每条微指令中完整存放下地址。

七、异常和中断机制 重点

1. 异常与中断的基本概念

异常：CPU执行指令过程中，由内部事件或指令执行情况引起的特殊控制转移。

中断：通常指由CPU外部事件引起的控制转移。

常见情况：

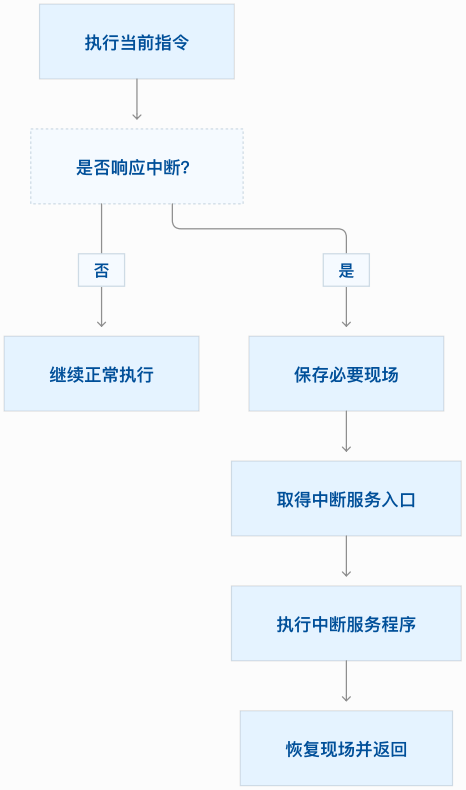
- 外部设备输入完成
- 定时器请求
- 除零异常
- 非法指令
- 缺页异常

2. 分类

类型	典型例子
外部中断	I/O完成、定时器中断
故障 Fault	缺页异常
陷阱 Trap	系统调用、断点
终止 Abort	严重硬件错误等

具体分类依体系结构而异。

3. 中断处理过程



必须记住的知识点

- CPU通常在指令执行结束的特定时刻检查并响应可屏蔽中断。
- 异常可能在指令执行过程中被检测到。
- 中断响应过程通常需要保存返回地址和必要状态。
- 中断服务程序可能还要保存通用寄存器等现场。
- 中断返回通常使用专门的返回机制。

4. 中断向量

中断向量一般用于确定中断服务程序入口。

中断向量地址与**中断服务程序入口地址**不要混淆。

例如：若某体系结构规定中断向量表中每项占4B，起始地址为1000H，第5号中断对应的向量项地址为：

$$1000_H + 5 \times 4$$

$$\boxed{1014_H}$$

该位置存储的信息可能用于取得实际中断处理程序入口。

具体是否直接存储入口地址，取决于体系结构。

八、指令流水线 必会

这是CPU章节最重要的计算题部分之一。

1. 流水线基本概念

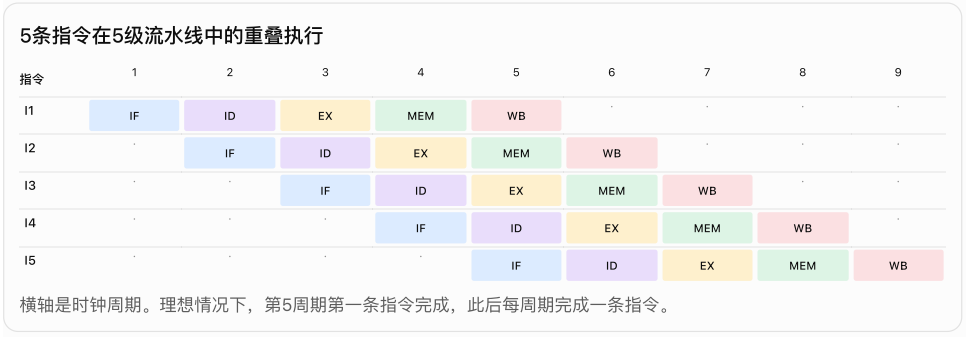
流水线的核心思想：

将一条指令的执行划分为多个阶段，使多条指令在不同阶段同时执行。

经典五级流水线：

阶段	英文	功能
IF	Instruction Fetch	取指
ID	Instruction Decode	译码、读寄存器
EX	Execute	运算或计算有效地址
MEM	Memory Access	数据存储器访问
WB	Write Back	写回结果

2. 五级流水线执行时间线



3. 流水线时钟周期

假设各阶段延迟分别为：

- IF：100 ns
- ID：80 ns
- EX：120 ns
- MEM：100 ns
- WB：60 ns

流水线时钟周期由最慢阶段决定：

$$\boxed{T_{\text{clk}} = \max(T_1, T_2, \ldots, T_k)}$$

因此：

$$T_{\text{clk}} = 120ns$$

如果每个流水段寄存器还增加10ns开销：

$$T_{\text{clk}} = 120 + 10$$

$$\boxed{130ns}$$

高频易错点

流水线时钟周期不是各阶段延迟之和，而是取最大值并考虑流水段寄存器开销。

九、流水线性能计算 必会

1. 非流水线执行时间

假设：

- 每条指令有 k 个阶段。
- 每阶段耗时 T。
- 共执行 n 条指令。

若串行执行：

$$\boxed{T_{\text{非流水}} = nkT}$$

2. 流水线执行时间

理想情况下：

$$\boxed{T_{\text{流水}} = (k + n - 1)T}$$

解释：

- 前 k 个周期用于填满流水线并完成第一条指令。
- 后续 $n - 1$ 条指令，每周期完成一条。

3. 流水线加速比

$$\boxed{S = \frac{T_{\text{非流水}}}{T_{\text{流水}}}}$$

代入：

$$\boxed{S = \frac{nk}{k + n - 1}}$$

当：

$$n \to \infty$$

则：

$$S \to k$$

所以理想情况下，k级流水线的最大加速比趋近于k。

注意：这里假设非流水线与流水线各阶段耗时相同，并忽略额外开销。

4. 经典例题

某CPU采用5级流水线，每级耗时100ns，执行100条指令。

求：

1. 非流水执行时间
2. 流水执行时间
3. 加速比

非流水：

$$T_1 = 100 \times 5 \times 100$$

$$T_1 = 50000ns$$

流水：

$$T_2 = (5 + 100 - 1) \times 100$$

$$T_2 = 10400ns$$

加速比：

$$S = \frac{50000}{10400}$$

$$\boxed{S \approx 4.81}$$

十、流水线吞吐率与效率 重点

1. 吞吐率

单位时间完成的指令数量。

$$\boxed{TP = \frac{n}{T_{总}}}$$

理想流水线：

$$TP = \frac{n}{(k + n - 1)T}$$

当指令数量足够多：

$$\boxed{TP_{\max} = \frac{1}{T}}$$

这里假设每周期能够完成一条指令。

2. 流水线效率

理想等长流水线中：

$$\boxed{E = \frac{n}{k + n - 1}}$$

如果流水线有5级、执行100条指令：

$$E = \frac{100}{104}$$

$$\boxed{E \approx 96.15\%}$$

注意：这是基于理想等长流水段模型得到的效率。对于不等长流水段或存在停顿的流水线，需要按照实际利用情况计算。

3. CPU执行时间公式

408经常将CPI与流水线结合起来考查。

$$\boxed{T_{\text{CPU}} = IC \times CPI \times T_{\text{clk}}}$$

也可以写成：

$$\boxed{T_{\text{CPU}} = \frac{IC \times CPI}{f}}$$

其中：

- IC：执行的指令条数
- CPI：平均每条指令需要的时钟周期数
- f：CPU时钟频率

平均CPI：

$$\boxed{CPI = \sum_{i=1}^{m} P_i CPI_i}$$

其中：

- P_i：第i类指令占比
- CPI_i：第i类指令的CPI

十一、流水线冒险 必会

流水线主要存在三种冒险：

1. 结构冒险
2. 数据冒险
3. 控制冒险

1. 结构冒险

多个指令同时争用同一个硬件资源。

例如：

- IF阶段需要访问存储器取指。
- MEM阶段需要访问存储器读取数据。
- 只有一个不能同时处理两次访问的存储器端口。

于是产生冲突。

解决方法：

- 增加硬件资源。
- 分离指令Cache和数据Cache。
- 插入流水线停顿。

2. 数据冒险

后一条指令依赖前一条指令的数据。

最典型的是RAW：

Read After Write，写后读相关。

例如：

```asm

```

```
ADD R1, R2, R3
SUB R4, R1, R5
```

第一条：

$$R1 \leftarrow R2 + R3$$

第二条：

$$R4 \leftarrow R1 - R5$$

第二条指令需要第一条指令生成的R1。

如果结果尚未准备好，就出现数据冒险。

三种数据相关

类型	中文名称	关系
RAW	写后读	后面的指令要读前面写入的数据
WAR	读后写	后面的指令写前面尚未读取的数据
WAW	写后写	两条指令写同一个位置

特别注意：

经典五级、按序发射且按序完成的简单流水线主要面临RAW数据冒险；WAR和WAW通常出现在更复杂的执行安排中。

3. 数据冒险解决方法

方法①：暂停流水线（Stall）

插入气泡Bubble，等待数据准备完成。

方法②：数据前递（Forwarding）

将ALU或后续阶段产生的数据直接转发给下一条需要该数据的指令。

例如：

```asm
ADD R1, R2, R3
SUB R4, R1, R5
```

如果支持前递，SUB可以较早获得ADD产生的R1，而不必等待完整写回。

方法③：指令调度

通过改变指令执行顺序，在相关指令之间安排其他独立指令。

十二、Load-Use数据冒险 必会

这是408流水线中很有代表性的题型。

例如：

```asm
LW  R1, 0(R2)
ADD R3, R1, R4
```

第一条：

$$R1 \leftarrow M[R2]$$

第二条：

$$R3 \leftarrow R1 + R4$$

问题：

LW通常要到MEM阶段后才能得到加载数据。

紧随其后的ADD在EX阶段就需要R1。

在经典五级流水线模型中，即使支持常规数据前递，通常仍需要暂停1个周期。

Load-Use 冒险：插入一个气泡

指令	1	2	3	4	5	6	7
LW	IF	ID	EX	MEM	WB	·	·
ADD	·	IF	ID	等待	EX	MEM	WB

示意表仅突出ADD的执行等待；真实流水线在停顿期间还需冻结相关前级流水寄存器并向后级插入气泡。

记忆结论

- 普通ALU指令之间的RAW相关，可能通过前递消除停顿。
- 紧邻的Load-Use相关，在经典五级流水线中通常仍需1个停顿周期。
- 具体停顿周期数取决于题目给出的流水线结构、阶段时序和前递路径。

十三、控制冒险 必会

控制冒险主要由转移指令引起。

例如：

```asm
BEQ R1, R2, LABEL
```

CPU需要判断：

$$R1 = R2?$$

如果相等，跳转到LABEL。

否则继续执行下一条指令。

问题在于：

分支结果尚未确定时，CPU可能已经取入后续指令。

如果预测方向错误，就要清除错误路径上的指令。

1. 解决方法

- 暂停取指。
- 尽早确定分支结果。
- 分支预测。
- 推测执行。

2. 分支预测

两种基本形式：

- 静态预测
- 动态预测

常见动态预测方法：

- 1位预测器
- 2位饱和计数器预测器

2位饱和计数器

典型状态：

00	01	10	11
强不跳	弱不跳	弱跳转	强跳转

实际跳转：计数器加1（最大11）。实际不跳转：计数器减1（最小00）。最高位决定预测方向。

考试常见结论

- 预测错误会降低流水线性能。
- 分支预测正确率越高，控制冒险造成的开销通常越小。
- 分支预测不会从根本上消除控制相关性。

十四、流水线CPI计算 必会

这类题建议重点练。

1. 基本公式

理想单发射流水线：

$$CPI_{\text{理想}} \approx 1$$

考虑流水线停顿：

$$\boxed{CPI_{\text{实际}} \approx 1 + \text{平均每条指令停顿周期数}}$$

如果题目给出多类停顿且影响可以相加：

$$\boxed{CPI = 1 + \sum P_i S_i}$$

其中：

- P_i：发生某类停顿的指令比例
- S_i：该类事件平均额外停顿周期数

对于有限条指令，精确计算还可能需要考虑流水线填充与排空时间。

2. 经典例题

某CPU采用理想CPI为1的流水线。

已知：

- 20%的指令会发生1周期数据停顿。
- 10%的指令发生分支预测错误。
- 每次分支预测错误额外损失3个周期。
- 两类开销可以直接累加。

求实际CPI。

数据停顿：

$$0.2 \times 1 = 0.2$$

分支预测错误开销：

$$0.1 \times 3 = 0.3$$

所以：

$$CPI = 1 + 0.2 + 0.3$$

$$\boxed{CPI = 1.5}$$

固定步骤

1. 找到理想CPI。
2. 每类事件的比例乘以惩罚周期。
3. 将各类平均开销相加。
4. 得到实际CPI。

十五、多处理器基本概念 重点

1. SISD、SIMD、MISD、MIMD

Flynn分类法：

类型	全称	含义
SISD	Single Instruction Single Data	单指令流、单数据流
SIMD	Single Instruction Multiple Data	单指令流、多数据流
MISD	Multiple Instruction Single Data	多指令流、单数据流
MIMD	Multiple Instruction Multiple Data	多指令流、多数据流

重点区分

SIMD

- 同一条指令作用于多个数据元素。
- 适合向量运算、图像处理等。

MIMD

- 多个处理单元可以执行不同指令流。
- 多核处理器和多处理器系统通常属于这一类。

2. 多核处理器

一块芯片上集成多个处理器核心。

特点：

- 多个核心可以并行执行任务。
- 核心之间可能共享Cache和主存。
- 需要考虑数据一致性与同步。

3. 硬件多线程

一个处理器核心保存多个线程的体系结构状态，并以一定方式调度它们执行。

常见：

- 细粒度多线程
- 粗粒度多线程
- 同时多线程 SMT

注意：

多核与多线程不是同一个概念。

多核是多个物理核心；硬件多线程是一个核心支持多个硬件线程上下文。

十六、408中央处理器计算题型汇总

题型	核心公式或方法	优先级
时钟周期	$T = 1/f$	必会
CPU执行时间	$T = IC \times CPI / f$	必会
平均CPI	$CPI = \sum P_i CPI_i$	必会
流水线时钟周期	$\max(T_i) + T_{寄存器}$	必会
理想流水线时间	$(k + n - 1)T$	必会
理想流水线加速比	$\frac{nk}{k+n-1}$	必会
吞吐率	$n/T_{总}$	重点
理想流水线效率	$\frac{n}{k+n-1}$	重点
微命令字段编码	$\lceil \log_2(m+1) \rceil$	必会
微地址字段	$\lceil \log_2 N \rceil$	重点
流水线停顿	逐周期分析数据产生与使用时刻	必会
分支预测CPI	理想CPI + 平均惩罚周期	必会

十七、考前速背：CPU必记20条

1. CPU主要由运算器和控制器组成。
2. PC保存下一条待取指令的地址。
3. IR保存当前指令。
4. MAR保存访存地址，MDR保存访存数据。
5. 取指操作通常为：PC→MAR→主存→MDR→IR。
6. 指令周期包含完成一条指令所需的全部操作。
7. 单总线结构同一时刻通常只能有一个数据源驱动总线。
8. 硬布线控制器通常速度较快，但修改较困难。
9. 微程序控制器通过微指令产生控制信号。
10. 水平型微指令通常较长，并行控制能力较强。
11. 垂直型微指令通常较短，编码较紧凑。
12. 典型五级流水线：IF、ID、EX、MEM、WB。
13. 流水线周期由最慢阶段及寄存器开销决定。
14. 理想流水线执行时间：$(k + n - 1)T$。
15. 流水线理想加速比上限趋近于流水段数k。
16. 结构冒险：硬件资源冲突。
17. 数据冒险：指令之间存在数据依赖。
18. 控制冒险：转移目标或方向尚未确定。
19. 经典五级流水线中，紧邻Load-Use通常暂停1周期。

20. CPU执行时间：$IC \times CPI / f$。

十八、基础自测：8道题

CPU章节基础巩固 0/8 已答

1. 程序计数器PC通常保存什么？

A. 当前指令内容

B. 下一条待取指令地址

C. 当前操作数

D. 主存读出的数据

2. 单总线数据通路中，哪个描述正确？

A. 多个寄存器可任意同时驱动总线

B. 一个时刻通常只有一个数据源驱动总线

C. ALU不需要输入操作数

D. 所有指令都只需一个节拍

3. 5级流水线，每级100ns，执行20条指令，理想执行时间是多少？

A. 2000ns

B. 2400ns

C. 2500ns

D. 10000ns

4. 哪一种属于数据冒险？

A. 多个指令争用存储器端口

B. 后续指令读取前序指令尚未写入的结果

C. 分支方向未知

D. 时钟频率过高

5. 经典五级流水线支持常规前递时，相邻LW与使用其结果的ADD通常要停顿几个周期？

A. 0

B. 1

C. 2

D. 5

6. CPU频率2GHz，平均CPI=2，执行10亿条指令需要多少秒？

A. 0.5秒

B. 1秒

C. 2秒

D. 4秒

7. 某微命令组有7个互斥命令，还需要一个不操作状态，字段至少多少位？

A. 2

B. 3

C. 4

D. 7

8. **哪一种描述符合典型水平型微指令？**

A. 编码紧凑但并行性弱

B. 微指令较长且并行性强

C. 必须只有一个微命令

D. 不使用控制存储器

最后：你目前应该怎么复习这一章？

我建议按照下面的顺序：

第一轮：先拿下计算题。

1. 流水线执行时间、加速比、吞吐率。
2. CPI与CPU执行时间。
3. 流水线冒险和停顿周期。
4. 微指令编码与字段位数。

第二轮：掌握数据通路分析。

重点练习给出CPU内部结构图，然后要求写出微操作、控制信号的题目。

第三轮：背概念。

包括：

- PC、IR、MAR、MDR的作用。
- 硬布线控制器与微程序控制器。
- 指令周期。
- 异常与中断。
- SIMD与MIMD。

你现在最需要的是把知识和题型建立联系。

如果一道题出现“5级流水线、各阶段延迟、指令数量”，立即想到 $(k+n-1)T$；如果出现“相关指令、数据前递”，立即画流水线时间线；如果出现“微命令分组”，立即想到 $\lceil \log_2(m+1) \rceil$。

只要能形成这种条件反射，这一章的基础题就比较容易稳定拿分。

408 计算机组成原理：存储系统复习笔记

存储系统是408计算机组成原理中非常重要的一章，尤其适合通过掌握固定公式和题型来拿分。

结合你目前“保基础分、猛攻常见题型”的复习策略，这份笔记按照 **核心概念 → 计算公式 → 典型例题 → 解题套路 → 易错点** 整理。

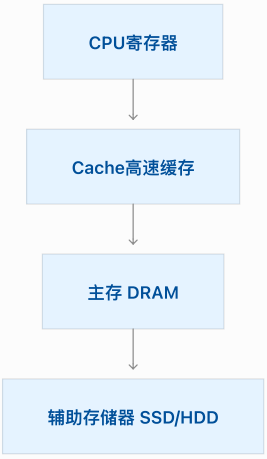
本章内容包括：

模块	主要考点	优先级
存储器基本概念	分类、性能指标、层次结构	重点
主存储器	SRAM、DRAM、ROM、编址方式	重点
主存扩展	位扩展、字扩展、芯片选择	必会
多模块存储器	交叉编址、流水访问、带宽	必会
Cache	映射方式、地址划分、命中率	必会
Cache替换策略	LRU、FIFO、随机替换	重点
Cache写策略	写直达、写回、写分配	重点
虚拟存储器	页式存储、TLB、地址转换	必会

一、存储系统基本概念

1. 存储系统的层次结构

计算机存储系统通常采用多级层次结构。



越靠近CPU：

- 访问速度通常越快。
- 容量通常越小。
- 单位存储成本通常越高。

越远离CPU则一般相反。

两个重要的存储层次

层次	主要解决的问题	管理方式
Cache—主存	CPU与主存的速度差异	主要由硬件自动管理
主存—辅存	主存容量不足	操作系统与硬件协同管理

记忆：Cache解决速度问题，虚拟存储器主要解决地址空间管理与主存容量限制问题。

2. 存储器的性能指标

存储容量

$$\boxed{\text{容量} = \text{存储单元数量} \times \text{每单元位数}}$$

例如：

$$1K \times 8 \text{ 位}$$

表示1024个存储单元，每个单元8位。

总容量：

$$1024 \times 8 = 8192bit = 1KiB$$

存取时间

存取时间 T_A：从发出存储器访问请求到完成该次访问所需的时间。

存取周期

存取周期 T_M：存储器连续进行两次独立访问所需的最小时间间隔。

通常：

$$\boxed{T_M \geq T_A}$$

两者不能混淆。

存储器带宽

单位时间传输的数据量。

理想情况下：

$$\boxed{BW = \frac{\text{每次传输数据量}}{\text{传输间隔}}}$$

如果每隔10ns传输8B：

$$BW = \frac{8B}{10ns} = 0.8GB/s$$

二、主存储器

1. SRAM与DRAM 重点

比较	SRAM	DRAM
中文	静态随机存储器	动态随机存储器
存储结构	典型为双稳态触发器	典型为电容存储电荷
是否需要刷新	不需要	需要
速度	通常较快	通常较慢
集成度	较低	较高
单位容量成本	较高	较低
典型用途	Cache	主存

重要结论：

- SRAM和DRAM都属于易失性存储器。
- DRAM需要定期刷新。
- DRAM典型读取具有破坏性，需要恢复数据。
- SRAM通常不需要刷新。

2. DRAM刷新

DRAM刷新是为了防止电容漏电导致数据丢失。

常见刷新方式：

方式	特点
集中刷新	集中一段时间逐行刷新，可能产生较长的刷新停顿
分散刷新	将刷新操作分散到各存储周期
异步刷新	在刷新周期内均匀安排刷新，减少连续停顿

典型计算题

某DRAM共有1024行，要求每2ms内全部刷新一次。

求平均每行刷新间隔。

$$T = \frac{2ms}{1024}$$

$$\boxed{T \approx 1.953\mu s}$$

注意：这是平均刷新调度间隔，并不是每行刷新操作本身所需的时间。

3. ROM的分类

类型	特点
MROM	掩模式ROM，制造时确定内容
PROM	一次性可编程
EPROM	可用紫外线擦除
EEPROM	可电擦除
Flash	基于电擦除技术，通常按块擦除

你做ESP32和STM32开发时接触的Flash，就属于非易失性存储器。

三、主存编址方式 必会

1. 按字节编址

每个地址对应1B。

例如32位机器字长：

$$32bit = 4B$$

按字节编址时，一个32位机器字占4个地址单位。

如果地址范围为：

$$0000H \sim FFFFH$$

则共有：

$$2^{16} = 65536$$

个地址。

总容量：

$$65536B = 64KiB$$

2. 按字编址

每个地址对应一个字。

例如：

- 字长32位。
- 地址线16根。
- 按字编址。

可寻址：

$$2^{16} = 65536$$

个字。

总容量：

$$65536 \times 4B$$

$$\boxed{256KiB}$$

必背公式

$$\boxed{\text{存储容量} = 2^{\text{地址位数}} \times \text{每地址对应字节数}}$$

四、存储芯片扩展 必会

这是非常固定的计算题型。

主要有：

1. 位扩展
2. 字扩展
3. 字位同时扩展

1. 位扩展

位扩展：增加每个存储字的位数。

例如：

用：

$$8K \times 4bit$$

芯片组成：

$$8K \times 16bit$$

需要：

$$\frac{16}{4} = 4$$

片。

$$\boxed{4片}$$

特点：

- 各芯片地址线并联。
- 各芯片同时选中。
- 数据线分别连接到数据总线的不同位。

2. 字扩展

字扩展：增加存储字的数量。

例如：

用：

$$8K \times 8bit$$

芯片组成：

$$32K \times 8bit$$

需要：

$$\frac{32K}{8K} = 4$$

片。

$$\boxed{4片}$$

特点：

- 数据线连接到相同的数据总线位。
- 通过片选信号选择不同存储芯片。
- 通常由高位地址经过译码产生片选信号。

3. 字位同时扩展

例如：

用：

$$8K \times 4bit$$

芯片组成：

$$32K \times 16bit$$

第一步：位扩展。

$$\frac{16}{4} = 4$$

第二步：字扩展。

$$\frac{32K}{8K} = 4$$

总芯片数量：

$$N = 4 \times 4$$

$$\boxed{16片}$$

通用公式

$$\boxed{N = \frac{\text{目标字数}}{\text{芯片字数}} \times \frac{\text{目标字长}}{\text{芯片位数}}}$$

前提是各扩展倍数为整数，且采用相同规格芯片。

五、地址线、数据线与片选 必会

1. 地址线数量

对于：

$$8K \times 8bit$$

芯片：

$$8K = 8192 = 2^{13}$$

因此需要：

$$\boxed{13根地址线}$$

数据线：

$$\boxed{8根}$$

固定公式

$$\boxed{\text{地址线数} = \log_2(\text{存储字数})}$$

$$\boxed{\text{数据线数} = \text{每个存储字的位数}}$$

2. 片选信号

假设：

- CPU有16根地址线。
- 使用4片 $16K \times 8bit$ 存储芯片。
- 总容量64KiB。

每片：

$$16K = 2^{14}$$

需要14根片内地址线：

$$A_0 \sim A_{13}$$

其余：

$$16-14=2$$

根高位地址线：

$$A_{14},A_{15}$$

用于选择芯片。

A15	A14	选中的芯片
0	0	第1片
0	1	第2片
1	0	第3片
1	1	第4片

做题口诀：低位片内寻址，高位用于片选。

六、多模块存储器 必会

多模块存储器通过多个存储体提高访问吞吐能力。

1. 高位交叉编址

高位地址用于选择存储体。

特点：

- 连续地址通常集中在同一存储体。
- 连续访问不容易实现存储体间的流水重叠。
- 适合按块访问等场景。

2. 低位交叉编址

低位地址用于选择存储体。

特点：

- 连续地址分布到不同存储体。
- 容易实现流水式连续访问。
- 可以提高连续访问带宽。

地址分布对比

假设有4个存储体。

连续字地址	高位交叉（示例：每体4字）	低位交叉
0	M0	M0
1	M0	M1
2	M0	M2
3	M0	M3
4	M1	M0
5	M1	M1
6	M1	M2
7	M1	M3

注意：高位交叉的具体分块大小取决于每个存储体的容量。

3. 低位交叉存储器模块数

假设：

- 每个存储体的存取周期为 T。
- 连续启动两个存储体的最小间隔为 τ。

为了在重新访问同一存储体前完成其上一次访问，需要：

$$m\tau \geq T$$

因此：

$$\boxed{m_{\min} = \left\lceil \frac{T}{\tau} \right\rceil}$$

其中：

- m：存储体数量
- T：存储体存取周期
- τ：连续启动间隔

典型例题

某存储体存取周期：

$$T = 120ns$$

连续启动间隔：

$$\tau = 30ns$$

求至少需要多少个存储体才能实现无冲突的连续流水访问。

$$m_{\min} = \frac{120}{30}$$

$$\boxed{m_{\min} = 4}$$

4. 连续读取n个字的时间

在理想低位交叉、模块数量足够且无额外冲突的条件下：

$$\boxed{T_{总} = T + (n-1)\tau}$$

例如连续读取8个字：

$$T_{总} = 120 + 7 \times 30$$

$$\boxed{330ns}$$

注意：若题目另给总线传输延迟、首次访问延迟或其他约束，应按题目时序计算。

七、Cache基本概念 必会

Cache利用程序的局部性原理，提高CPU访问存储器的平均速度。

1. 局部性原理

时间局部性

刚刚访问过的数据，在短时间内可能再次访问。

例如循环中的变量。

空间局部性

访问某个地址后，附近的地址也可能被访问。

例如连续遍历数组。

2. Cache基本术语

名称	含义
Cache行	Cache中存放一个主存块的单位
主存块	主存与Cache之间的数据传送单位
命中 Hit	所需数据已在Cache中
缺失 Miss	所需数据不在Cache中
命中率 h	访问命中的比例
缺失率	$1-h$

八、Cache映射方式 必会

三种：

1. 直接映射
2. 全相联映射
3. 组相联映射

1. 直接映射

一个主存块只能映射到一个固定的Cache行。

$$\boxed{\text{Cache行号} = \text{主存块号} \bmod \text{Cache行数}}$$

地址结构

标记 Tag	行号 Index	块内偏移 Offset

若：

- 主存地址长度：32位
- Cache数据容量：16KiB
- 块大小：64B

第一步：求块内偏移位数。

$$b = \log_2 64 = 6$$

第二步：求Cache行数。

$$L = \frac{16KiB}{64B} = 256$$

第三步：求行号位数。

$$r = \log_2 256 = 8$$

第四步：求标记位数。

$$t = 32 - 8 - 6$$

$$\boxed{t = 18}$$

最终地址结构：

Tag	Index	Offset
18位	8位	6位

2. 全相联映射

主存任意块可以放入Cache任意行。

地址结构：

Tag	Offset

仍然假设：

- 主存地址32位
- 块大小64B

则：

$$b = 6$$

$$t = 32 - 6$$

$$\boxed{t = 26}$$

全相联映射不需要利用主存地址中的固定行号字段来选择Cache行。

3. 组相联映射

Cache划分为多个组，每组包含若干行。

主存块只能进入指定组，但可以放在该组任意一行。

$$\boxed{\text{组号} = \text{主存块号} \bmod \text{组数}}$$

例如4路组相联：

- 每组4行。
- 一个主存块可以放入对应组中的任意一行。

地址结构

Tag	组号 Index	Offset

经典例题

已知：

- 主存地址32位。
- Cache容量16KiB。
- 块大小64B。
- 采用4路组相联。

求Tag、组号和块内偏移位数。

第一步：求Cache行数。

$$L = \frac{16KiB}{64B} = 256$$

第二步：求组数。

$$G = \frac{256}{4} = 64$$

第三步：求组号位数。

$$g = \log_2 64 = 6$$

第四步：求块内偏移。

$$b = \log_2 64 = 6$$

第五步：求Tag。

$$t = 32 - 6 - 6$$

$$\boxed{t = 20}$$

最终：

Tag	Index	Offset
20位	6位	6位

九、Cache地址划分万能公式

假设：

- 主存地址宽度：m 位
- Cache数据容量：C 字节
- 块大小：B 字节
- 相联度：k

则：

$$\boxed{L = \frac{C}{B}}$$

$$\boxed{G = \frac{L}{k} = \frac{C}{Bk}}$$

块内偏移位数：

$$\boxed{b = \log_2 B}$$

组号位数：

$$\boxed{g = \log_2 G}$$

标记位数：

$$\boxed{t = m - g - b}$$

这些公式适用于按字节编址、参数均为适当的2的整数次幂的常规题型。

三种映射统一理解

映射方式	相联度k	组数
直接映射	1	Cache总行数
全相联映射	Cache总行数	1
k路组相联	k	Cache总行数÷k

因此：

直接映射和全相联映射，都可以看作组相联映射的特殊情况。

十、Cache实际容量计算 必会

这是408非常容易设置陷阱的地方。

题目给出的Cache容量，通常指**数据区容量**。

但实际Cache还需要保存：

- 标记Tag
- 有效位Valid
- 脏位Dirty（若采用写回）
- 替换策略信息等

典型例题

某Cache：

- 数据容量16KiB
- 块大小64B
- 主存地址32位
- 直接映射
- 每行有1位有效位、1位脏位
- 忽略其他开销

求Cache总存储位数。

已知：

$$L = \frac{16KiB}{64B} = 256$$

Tag为：

$$t = 18$$

每行数据：

$$64 \times 8 = 512bit$$

每行总位数：

$$512 + 18 + 1 + 1 = 532bit$$

Cache总位数：

$$256 \times 532$$

$$\boxed{136192bit}$$

记住：求Cache总容量时，不能漏掉Tag及控制位。

十一、Cache命中率与平均访问时间 必会

1. 命中率

$$\boxed{h = \frac{\text{命中次数}}{\text{总访问次数}}}$$

缺失率：

$$\boxed{r = 1 - h}$$

2. 平均访问时间

需要看题目如何定义缺失时间。

情况①：缺失时间包含完整的缺失访问过程

若：

- 命中耗时 T_c
- 缺失时总耗时 T_m

则：

$$\boxed{T_{\text{avg}} = hT_c + (1-h)T_m}$$

情况②：题目给出缺失额外惩罚

如果：

- 每次访问首先查询Cache，耗时 T_c
- 未命中时额外增加 P

则：

$$\boxed{T_{\text{avg}} = T_c + (1-h)P}$$

例题

Cache命中率95%，Cache访问时间10ns，缺失时额外增加100ns。

求平均访问时间。

$$T_{\text{avg}} = 10 + 0.05 \times 100$$

$$\boxed{15ns}$$

注意：

一定要区分“缺失总耗时”和“缺失额外开销”。

十二、Cache替换算法 重点

当Cache对应组已满，需要替换其中某一行。

常见算法：

1. FIFO

先进先出。

替换最早进入该组的块。

2. LRU

最近最少使用。

替换最长时间没有被访问的块。

3. 随机替换

随机选择某个Cache行替换。

LRU经典题型

假设Cache只有3行，采用全相联映射和LRU。

访问序列：

$$1, 2, 3, 1, 4$$

逐步分析：

访问	Cache内容（左为最近使用）	命中情况
1	1	缺失
2	2,1	缺失
3	3,2,1	缺失
1	1,3,2	命中
4	4,1,3	缺失，替换2

最终：

$$\boxed{\text{Cache中有 } 4, 1, 3}$$

LRU记忆口诀

> 谁最久没用，就替换谁。

注意：LRU不是替换最早进入Cache的块，那是FIFO。

十三、Cache写策略 重点

Cache写策略分为两个独立维度。

1. 写命中策略

写直达（Write Through）

写Cache的同时写主存。

优点：

- 主存与Cache中的对应数据较容易保持一致。

缺点：

- 写入主存次数较多。
- 可能需要写缓冲区提高性能。

写回（Write Back）

写操作先修改Cache。

被修改的Cache块被替换时，再写回下一级存储器。

通常需要脏位：

$$Dirty = 1$$

表示Cache块已被修改。

2. 写缺失策略

写分配（Write Allocate）

写缺失时，先把对应主存块调入Cache，再修改。

非写分配（No Write Allocate）

写缺失时，不将该块调入Cache，而是直接向下一级存储器执行写操作。

常见搭配

策略组合	特点
写回 + 写分配	常见
写直达 + 非写分配	常见
写直达 + 写分配	也可以实现
写回 + 非写分配	不常见，但不能简单认定不可能

十四、虚拟存储器 必会

1. 基本概念

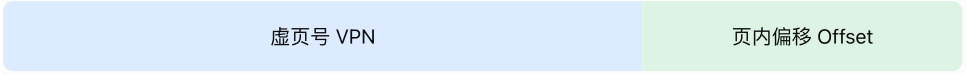
虚拟存储器通过地址映射和按需调入机制，使程序能够使用虚拟地址空间，而不必要求其全部内容始终驻留在主存。

主要涉及：

- CPU
- MMU
- 页表
- TLB
- 主存
- 辅存

2. 虚拟地址结构

典型页式虚拟存储：



如果页面大小为：

$$4KiB = 2^{12}B$$

则页内偏移占：

$$\boxed{12\text{ 位}}$$

若虚拟地址32位：

$$VPN = 32 - 12$$

$$\boxed{20\text{ 位}}$$

3. 物理地址结构

物理页框号 PPN	页内偏移 Offset

注意：

虚拟地址和物理地址的页内偏移相同，改变的是页号。

4. 虚拟地址转换

例如：

- 虚拟地址：0x12345
- 页面大小：4KiB

页内偏移：

$$Offset = 0x345$$

虚页号：

$$VPN = 0x12$$

假设页表查得对应物理页框号：

$$PPN = 0x8$$

则物理地址：

$$PA = (0x8 \ll 12) + 0x345$$

$$\boxed{PA = 0x8345}$$

十五、页表与TLB 必会

1. 页表

页表保存虚拟页号与物理页框号的映射关系。

页表项通常包含：

- 物理页框号
- 有效位/存在位
- 访问权限
- 修改位
- 访问位等

具体字段取决于体系结构。

2. TLB

TLB：快表，保存近期使用过的地址转换信息。

作用：

加速虚拟地址到物理地址的转换。

TLB与Cache不要混淆。

TLB	Cache
缓存地址转换信息	缓存数据或指令
加快地址翻译	加快数据访问
未命中通常需要查页表	未命中通常访问下一级存储
命中不代表数据在Cache中	命中不代表无需TLB地址翻译

十六、TLB平均访问时间 必会

假设：

- TLB查询时间：t
- 主存访问时间：m
- TLB命中率：h
- 页表为单级页表
- 不考虑Cache和缺页
- TLB查询与主存访问串行

TLB命中：

$$T_{\text{hit}} = t + m$$

TLB未命中：

$$T_{\text{miss}} = t + 2m$$

因为需要：

1. 查询TLB。
2. 访问主存中的页表。
3. 访问目标数据。

平均时间：

$$\boxed{T_{\text{avg}} = h(t+m) + (1-h)(t+2m)}$$

整理：

$$\boxed{T_{\text{avg}} = t + (2-h)m}$$

例题

已知：

- TLB查询10ns
- 主存访问100ns
- TLB命中率90%

则：

$$T_{\text{avg}} = 10 + (2 - 0.9) \times 100$$

$$\boxed{120ns}$$

注意：对于多级页表、Cache或并行查询情况，公式需要相应调整。

十七、多级页表计算 必会

这是408经常出现的综合题型。

1. 经典例题

假设：

- 虚拟地址32位。
- 页面大小4KiB。
- 页表项大小4B。
- 采用二级页表。
- 每级页表恰好占一页。

求：

1. 页内偏移位数。
2. 二级页表索引位数。
3. 一级页表索引位数。

第一步：页内偏移

$$4KiB = 2^{12}B$$

所以：

$$\boxed{12\text{ 位}}$$

第二步：每页能容纳多少页表项

$$\frac{4KiB}{4B} = 1024$$

$$1024 = 2^{10}$$

所以二级页表索引：

$$\boxed{10\text{ 位}}$$

第三步：一级页表索引

$$32 - 12 - 10 = 10$$

所以：

$$\boxed{10\text{ 位}}$$

虚拟地址结构：

一级页号	二级页号	页内偏移
10位	10位	12位

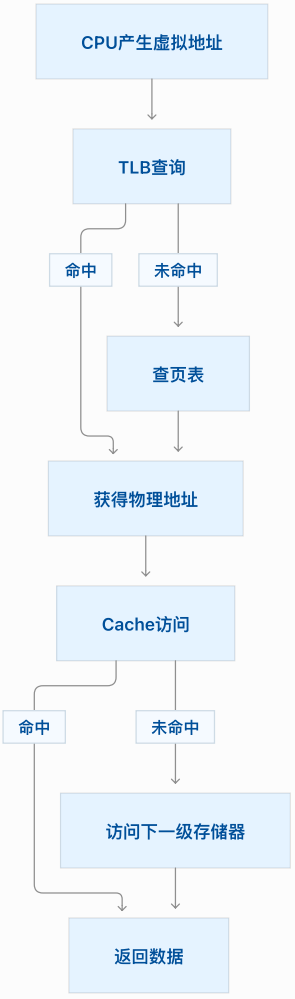
固定步骤

1. 页面大小取$\log_2$，求页内偏移。
2. 页面大小÷页表项大小，求每页页表项数。
3. 对每页页表项数取$\log_2$，求对应级索引位数。
4. 用虚拟地址总位数减去已知字段，求剩余字段。

十八、Cache与TLB联合地址转换

这部分是综合题重点。

典型访问过程：



注意：这是便于理解的“先地址转换、后物理地址Cache访问”模型。

实际处理器也可能采用VIPT等结构，允许部分Cache索引过程与TLB查询并行。

高频易错点

1. TLB命中不一定Cache命中。
2. TLB未命中不一定发生缺页。
3. 缺页异常需要操作系统参与。
4. Cache缺失通常由硬件自动处理。
5. 页表一般存放在主存中，并可能被Cache缓存。

十九、存储系统题型速查表

题型关键词	立即想到
芯片数量、扩展容量	字扩展×位扩展
地址线、数据线	$\log_2$ 字数、字长
存储体数量	$\lceil T/\tau \rceil$
连续访问n个字	$T+(n-1)\tau$
Cache地址划分	Tag、Index、Offset
Cache组数	$C/(B\times k)$
Cache实际容量	数据位+Tag+控制位
Cache命中率	$h=$ 命中次数/总次数
平均访问时间	命中耗时×命中率+缺失耗时×缺失率
虚拟页号	虚拟地址÷页面大小
页内偏移	虚拟地址对页面大小取余
页表项数量	页表大小÷页表项大小
TLB平均时间	命中时间与未命中时间加权

二十、考前速背：存储系统20条

1. SRAM不需要刷新，DRAM需要刷新。
2. SRAM通常用于Cache，DRAM通常用于主存。
3. Cache利用程序的时间局部性和空间局部性。
4. 地址线数由存储字数决定，数据线数由字长决定。
5. 位扩展增加字长，字扩展增加字数。
6. 存储芯片数量=字扩展倍数×位扩展倍数。
7. 低位交叉编址有利于连续地址的流水访问。
8. 连续流水访问的存储体数量至少为 $\lceil T/\tau \rceil$。
9. 直接映射：主存块号对Cache行数取模。
10. 组相联映射：主存块号对Cache组数取模。
11. 全相联映射：主存块可进入任意Cache行。
12. Cache行数=数据容量÷块大小。
13. 组数=Cache行数÷相联度。
14. 块内偏移位数=$\log_2 B$。
15. Tag位数=主存地址位数−组号位数−块内偏移位数。
16. LRU替换最长时间未使用的块。
17. 写回策略通常需要脏位。
18. 页内偏移位数=$\log_2$ 页面大小。
19. TLB缓存地址转换信息，Cache缓存数据或指令。
20. TLB未命中不等于缺页，Cache未命中也不等于缺页。

二十一、基础自测：8道题

存储系统基础巩固 0/8 已答

1. 8K×4位的芯片组成32K×16位存储器，共需多少片？

A. 4

B. 8

C. 12

D. 16

2. 某Cache为16KiB、块64B、4路组相联，组号需要多少位？

A. 4

B. 6

C. 8

D. 10

3. 32位地址、直接映射、Cache为16KiB、块64B，Tag是多少位？

A. 16

B. 18

C. 20

D. 26

4. 4个存储体，存取周期120ns，连续启动间隔30ns，读取8个连续字的理想耗时？

A. 240ns

B. 300ns

C. 330ns

D. 960ns

5. Cache命中率90%，Cache查询10ns，缺失额外增加100ns，平均访问时间是多少？

A. 10ns

B. 20ns

C. 100ns

D. 110ns

6. 4KiB页面，虚拟地址32位，页内偏移多少位？

A. 10

B. 12

C. 16

D. 20

7. 哪一种说法正确？

A. TLB未命中必然缺页

B. Cache未命中必然缺页

C. TLB保存近期地址映射信息

D. DRAM不需要刷新

8. LRU算法会优先替换什么块？

A. 最早进入的块

B. 最近使用的块

C. 最长时间未使用的块

D. 地址最大的块

最后：这一章的拿分顺序

按你现在希望“遇到题就能识别公式、照步骤做出来”的目标，我建议：

1. **Cache地址结构**：先练到看到容量、块大小、相联度就能求Tag、Index、Offset。
2. **存储器扩展**：重点熟练字扩展、位扩展、地址线和片选。
3. **多模块存储器**：掌握模块数量和连续访存时间。
4. **TLB与页表**：掌握虚实地址划分、多级页表、平均访问时间。
5. **Cache命中率与替换算法**：熟练计算命中次数、访存时间。
6. **SRAM/DRAM、写策略**：通过选择题巩固概念。

这六类题型值得反复练习。尤其是前三类，计算步骤通常比较固定，很适合作为你的基础分突破口。

408 计算机组成原理：数据的表示和运算（考研复习笔记）

这一章是408计算机组成原理中的重要计算章节，特点是**公式多、规则多，但常见题型比较固定**。

尤其是原码、反码、补码、移位、溢出判断、定点数运算、浮点数运算等内容，掌握固定步骤后，比较容易在考试中拿到基础分。

这份笔记按照 **基本概念 → 核心公式 → 典型例题 → 解题步骤 → 易错点** 整理，重点放在能够直接用于解题的知识上。

本章考点总览

知识模块	主要考点	优先级
数制与编码	进制转换、BCD、ASCII	重点
定点数表示	原码、反码、补码、移码	必会
数据类型与转换	有符号、无符号、扩展、截断	必会
移位运算	算术移位、逻辑移位	必会
定点加减法	补码运算、溢出判断	必会
定点乘除法	原码乘法、补码乘法、除法原理	重点
浮点数	IEEE 754、规格化、阶码	必会
浮点数运算	对阶、尾数运算、舍入、溢出	必会
运算器	ALU、加法器、进位链	重点

一、数制与进制转换

1. 常见进制

进制	基数	使用数字	示例
二进制	2	0、1	1011_2
八进制	8	0～7	13_8
十进制	10	0～9	11_{10}
十六进制	16	0～9、A～F	B_{16}

重要对应关系

$$\boxed{1\text{ 位八进制} = 3\text{ 位二进制}}$$

$$\boxed{1\text{ 位十六进制} = 4\text{ 位二进制}}$$

例如：

$$(10110110)_2$$

按4位分组：

$$1011 \quad 0110$$

对应：

$$\boxed{B6_H}$$

2. 二进制转十进制

整数部分按权展开：

$$(101101)_2$$

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1$$

$$= 32 + 8 + 4 + 1$$

$$\boxed{45}$$

小数部分：

$$(0.101)_2$$

$$= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$= 0.5 + 0.125$$

$$\boxed{0.625}$$

3. 十进制转二进制

整数部分：**除2取余，倒序排列。**

小数部分：**乘2取整，顺序排列。**

例如：

$$0.625 \times 2 = 1.25$$

取1，剩余0.25。

$$0.25 \times 2 = 0.5$$

取0。

$$0.5 \times 2 = 1.0$$

取1。

因此：

$$\boxed{0.625_{10} = 0.101_2}$$

注意：部分十进制小数不能用有限位二进制精确表示，例如0.1。

二、原码、反码、补码、移码 必会

这是全章必须掌握的基础。

1. 原码

原码的规则：

- 最高位为符号位。
- 0表示正数。
- 1表示负数。
- 其余位表示数值的绝对值。

例如用8位表示：

$$+5 = 00000101$$

$$-5 = 10000101$$

n位原码表示范围

$$\boxed{-(2^{n-1}-1) \sim +(2^{n-1}-1)}$$

8位原码：

$$\boxed{-127 \sim 127}$$

原码存在两个零：

$$+0 = 00000000$$

$$-0 = 10000000$$

2. 反码

正数反码与原码相同。

负数反码：

符号位不变，数值位按位取反。

例如：

$$-5_{原} = 10000101$$

符号位1不变，其余取反：

$$\boxed{-5_{反} = 11111010}$$

反码也存在正零和负零。

3. 补码

正数补码与原码相同。

负数补码：

原码符号位不变、数值位取反，再加1。

更通用的理解是：

$$\boxed{[-x]_{补} = 2^n - x}$$

这里 $x > 0$，且结果按照n位二进制编码。

例如8位补码表示 -5：

$$2^8 - 5 = 251$$

$$251 = 11111011_2$$

所以：

$$\boxed{[-5]_{补} = 11111011}$$

8位补码范围

$$\boxed{-128 \sim 127}$$

一般n位补码范围：

$$\boxed{-2^{n-1} \sim 2^{n-1} - 1}$$

补码只有一个零：

$$00000000$$

最容易考的特殊值

8位补码：

$$10000000 = -128$$

而不是负零。

$$11111111 = -1$$

$$01111111 = 127$$

4. 补码快速求真值

例如8位补码：

$$11110110$$

最高位为1，是负数。

方法一：取反加1。

$$11110110$$

取反：

$$00001001$$

加1：

$$00001010 = 10$$

所以真值：

$$\boxed{-10}$$

方法二：直接减去 2^n。

$$11110110_2 = 246$$

$$246 - 256 = -10$$

因此：

$$\boxed{x = -10}$$

考试推荐第二种方法，尤其适合十六进制补码。

例如：

$$FC_H = 252$$

按8位补码解释：

$$252 - 256 = -4$$

5. 移码

移码通常用于表示浮点数的阶码。

常见定义：

$$\boxed{[x]_{\text{移}} = x + 2^{n-1}}$$

其中n是编码总位数，此时偏置值为 2^{n-1}。

例如用8位、偏置128的移码表示 -5：

$$-5 + 128 = 123$$

$$\boxed{01111011}$$

重要区别

IEEE 754浮点数阶码也采用偏置编码，但常用偏置不是 2^{n-1}，而是：

$$\boxed{Bias = 2^{k-1} - 1}$$

例如单精度8位阶码：

$$Bias = 127$$

不要把教材中的普通移码公式与IEEE 754阶码直接混为一谈。

三、四种机器数表示方法对比

以8位表示 $+5$ 和 -5：

编码	+5	-5
原码	00000101	10000101
反码	00000101	11111010
补码	00000101	11111011
移码（偏置128）	10000101	01111011

n位整数表示范围

编码	最小值	最大值	零的数量
原码	$-(2^{n-1}-1)$	$2^{n-1}-1$	2
反码	$-(2^{n-1}-1)$	$2^{n-1}-1$	2
补码	-2^{n-1}	$2^{n-1}-1$	1
移码（偏置 2^{n-1}）	-2^{n-1}	$2^{n-1}-1$	1

互动练习：8位补码转换

观察负数的机器数编码

-5（十进制）

原码	10000101
反码	11111010
补码	11111011
移码（偏置128）	01111011

范围限制为 -127～127，以便四种编码都能表示。这里使用固定8位整数格式。

四、有符号数与无符号数 必会

相同的二进制位模式，按照不同数据类型解释，数值可能完全不同。

例如：

$$11111111$$

如果按8位无符号整数解释：

$$\boxed{255}$$

如果按8位补码有符号整数解释：

$$\boxed{-1}$$

1. 表示范围

n位无符号整数：

$$\boxed{0 \sim 2^n - 1}$$

n位补码有符号整数：

$$\boxed{-2^{n-1} \sim 2^{n-1} - 1}$$

例如8位：

类型	范围
unsigned	0～255
signed（补码）	-128～127

2. 符号扩展

有符号补码从低位数扩展到高位数：

复制原来的最高符号位。

例如8位 -5：

$$11111011$$

扩展到16位：

$$\boxed{1111111111111011}$$

数值仍然是 -5。

3. 零扩展

无符号数扩展时，高位补0。

例如：

$$11111011$$

从8位零扩展到16位：

$$\boxed{0000000011111011}$$

表示251。

必背结论

- 有符号补码：符号扩展。
- 无符号数：零扩展。
- 截断高位可能造成数值改变。
- 相同位模式，类型不同，含义可能不同。

五、移位运算 必会

移位运算主要分为：

- 逻辑移位
- 算术移位
- 循环移位

1. 逻辑移位

通常用于无符号二进制数。

规则：

- 左移：低位补0。
- 右移：高位补0。

例如：

$$00110100$$

逻辑左移1位：

$$01101000$$

逻辑右移1位：

$$00011010$$

注意：移出的位会丢失，除非指令集另有保存规则。

2. 算术移位

通常用于有符号数。

对于补码：

- 算术左移：右侧补0，原有位整体左移。
- 算术右移：左侧补符号位。

例如8位补码：

$$-8 = 11111000$$

算术右移1位：

$$11111100$$

对应：

$$\boxed{-4}$$

算术右移2位：

$$11111110$$

对应：

$$\boxed{-2}$$

注意算术左移可能溢出

例如：

$$01000000 = 64$$

左移1位：

$$10000000$$

按8位补码解释：

$$-128$$

数学上的结果本应为128，但8位补码无法表示。

因此发生溢出。

3. 循环移位

循环移位将移出的位重新移入另一端。

例如：

$$10010110$$

循环左移1位：

$$00101101$$

逻辑右移与算术右移对比

原数据	逻辑右移1位	算术右移1位
01110000	00111000	00111000
11110000	01111000	11111000

记忆：逻辑右移补0，补码算术右移补符号位。

六、定点数加减运算 必会

1. 补码加法

使用补码时：

$$\boxed{[x+y]_{补} = [x]_{补} + [y]_{补}}$$

只保留规定字长的低n位。

例如8位：

$$5 + (-3)$$

$$5 = 00000101$$

$$-3 = 11111101$$

相加：

```
纯文本
  00000101
+ 11111101
```

```
----------
1 00000010
```

舍弃第9位进位：

$$00000010 = 2$$

结果：

$$\boxed{2}$$

2. 补码减法

减法转换为加法：

$$\boxed{x - y = x + (-y)}$$

例如：

$$5 - 3$$

实际计算：

$$5 + (-3) = 2$$

注意：

如果已经知道 $[y]_{补}$，求 $[-y]_{补}$ 通常可以对原n位编码**按位取反再加1**。

但对于最小负数，例如8位补码 -128，其相反数128无法用8位有符号数表示。

七、溢出判断 必会

这是408非常典型的选择题和计算题。

1. 什么是溢出?

当数学运算结果超出机器数能够表示的范围，就发生溢出。

8位补码范围：

$$[-128, 127]$$

例如：

$$100 + 50 = 150$$

150超出表示范围，所以发生溢出。

2. 符号判断法

对于补码加法：

两个同号数相加，结果符号相反，则发生溢出。

例如：

$$100 + 50$$

都是正数，但8位结果会变成负数。

所以溢出。

相反：

- 正数+负数，不会发生有符号加法溢出。
- 正数+正数，可能正溢出。
- 负数+负数，可能负溢出。

例题

8位补码：

$$01111111 + 00000001$$

即：

$$127 + 1$$

二进制结果：

$$10000000$$

解释为：

$$-128$$

因为正数加正数得到了负数，所以：

$$\boxed{OF = 1}$$

3. 双符号位法

使用变形补码，即两个符号位。

结论：

- 两个符号位相同：没有溢出。
- 两个符号位不同：发生溢出。

例如：

$$01\cdots$$

表示正溢出。

$$10\cdots$$

表示负溢出。

注意：这里讨论的是双符号位的变形补码运算，不是普通单符号位编码。

4. 进位判断法

设：

- C_{in}：进入符号位的进位。
- C_{out}：符号位向最高位之外产生的进位。

则：

$$\boxed{OF = C_{in} \oplus C_{out}}$$

其中：

$$\oplus$$

表示异或。

也就是说：

两个进位不同，则发生有符号溢出。

5. CF与OF的区别

标志	判断内容
CF	无符号加法进位或无符号减法借位
OF	有符号运算结果是否超出范围

例如8位：

$$255 + 1$$

无符号结果超出255：

$$CF = 1$$

而按补码解释：

$$-1 + 1 = 0$$

没有有符号溢出：

$$OF = 0$$

注意：减法中CF表示借位的具体约定依指令集而定。

八、无符号数比较与有符号数比较

相同的二进制结果，比较方式可能不同。

1. 无符号比较

通常根据：

- CF
- ZF

等标志判断。

2. 有符号比较

通常需要：

- SF
- OF
- ZF

例如对于某些常见ISA的减法比较结果：

$$x < y$$

有符号条件可以通过：

$$\boxed{SF \neq OF}$$

来判断。

无符号条件通常由借位相关标志判断。

注意：具体条件码定义需要以题目给定的指令集为准。

九、定点乘法 重点

定点乘法主要考查：

- 原码一位乘法
- 补码一位乘法
- 移位与加法
- 乘积位数

1. 乘法的基本原理

二进制乘法可以通过：

部分积相加与移位 实现。

例如：

$$1011_2 \times 0101_2$$

即：

$$11 \times 5$$

展开：

```
    纯文本
      1011
×     0101
-----------
      1011
     0000
    1011
   0000
-----------
    110111
```

$$110111_2 = 55$$

因此：

$$\boxed{11 \times 5 = 55}$$

2. 原码一位乘法

基本思想：

1. 符号位单独处理。
2. 数值部分进行无符号乘法。
3. 根据乘数最低位决定是否加被乘数。
4. 通过移位形成部分积。
5. 最后确定乘积符号。

乘积符号：

$$\boxed{S = S_x \oplus S_y}$$

同号得正，异号得负。

3. 补码一位乘法

常见采用Booth算法思想。

设：

- Q_0：乘数当前最低位。
- Q_{-1}：附加位。

按下面规则处理：

Q_0Q_{-1}	操作
00	不加不减
01	加被乘数
10	减被乘数
11	不加不减

每轮操作后执行算术右移。

记忆口诀

> 01加，10减，00和11不操作。

注意：具体考试题可能使用教材规定的Booth变体，运算寄存器宽度、循环次数及最后一次是否移位，需要严格遵循题目算法。

4. 乘积位数

两个n位无符号整数相乘，完整乘积最多需要：

$$\boxed{2n \text{ 位}}$$

两个n位补码有符号整数相乘，完整乘积也可以用2n位补码表示。

例如：

8位×8位：

$$\boxed{16 \text{ 位乘积}}$$

如果只保留低8位，就可能丢失高位信息。

十、定点除法 重点

定点除法主要考查：

- 恢复余数法
- 不恢复余数法（加减交替法）
- 商和余数的处理
- 除法溢出与除零

1. 恢复余数法

基本思想：

1. 将部分余数减去除数。
2. 判断余数符号。

3. 若余数非负，则商相应位置1。
4. 若余数为负，则恢复余数，商相应位置0。
5. 移位，继续下一轮。

2. 不恢复余数法

核心思想：

根据余数符号决定下一步加除数还是减除数，避免每次都立即恢复余数。

一般规则：

- 余数非负：下一步倾向于减除数。
- 余数为负：下一步倾向于加除数。

不同教材算法的移位、加减顺序略有区别，需要按照题目给出的规则执行。

3. 除法易错点

- 除数为0不能正常完成除法。
- 商可能超出规定机器数范围。
- 整数除法与小数定点除法的运算规则不完全相同。
- 余数符号、修正方式需要根据具体算法判断。

对于你目前以基础分为目标的复习，建议先理解乘除法的原理和运算表，再针对真题出现的具体算法练习竖式，不必一开始把大量不同变体混在一起。

十一、浮点数基本表示 必会

浮点数通常采用科学计数法思想。

一般表示为：

$$\boxed{N = (-1)^S \times M \times R^E}$$

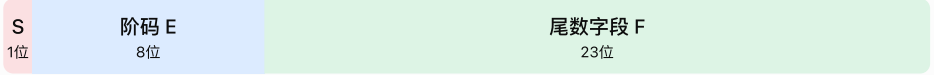
其中：

- S：符号。
- M：尾数。
- R：基数。
- E：阶码。

二进制浮点数通常以2为基数。

1. IEEE 754单精度

32位浮点数：



IEEE 754 binary32（单精度）字段结构

单精度：

- 符号位：1位
- 阶码位：8位
- 尾数字段：23位
- 阶码偏置值：127

对于规格化有限数：

$$\boxed{N = (-1)^S \times (1.F)_2 \times 2^{E-127}}$$

注意：这里的E是阶码字段按无符号整数解释后的值，且 $1 \leq E \leq 254$。

2. IEEE 754双精度

64位浮点数：

字段	位数
符号位	1
阶码	11
尾数字段	52

偏置值：

$$\boxed{1023}$$

规格化有限数：

$$\boxed{N = (-1)^S \times (1.F)_2 \times 2^{E-1023}}$$

单精度与双精度对比

项目	单精度	双精度
总位数	32	64
阶码位数	8	11
尾数字段位数	23	52
有效精度位数（含隐藏位）	24	53
Bias	127	1023

十二、IEEE 754浮点数转换 必会

这是最值得掌握的固定题型之一。

典型例题：将13.25转成IEEE 754单精度

第一步：确定符号位

13.25是正数：

$$S = 0$$

第二步：转换为二进制

整数部分：

$$13 = 1101_2$$

小数部分：

$$0.25 = 0.01_2$$

所以：

$$13.25 = 1101.01_2$$

第三步：规格化

$$1101.01_2$$

移动小数点：

$$\boxed{1.10101_2 \times 2^3}$$

真实阶码：

$$e = 3$$

第四步：计算阶码字段

$$E = e + 127$$

$$E = 3 + 127 = 130$$

130转二进制：

$$\boxed{10000010}$$

第五步：计算尾数字段

规格化尾数：

$$1.10101$$

隐藏最高位1，只保存小数部分：

$$10101$$

补齐23位：

$$\boxed{10101000000000000000000}$$

第六步：拼接

$$S = 0$$

$$E = 10000010$$

$$F = 10101000000000000000000$$

最终：

```
纯文本
0 | 10000010 | 10101000000000000000000
```

二进制整体：

```
纯文本
01000001010101000000000000000000
```

对应十六进制：

$$\boxed{41540000_H}$$

固定步骤

遇到十进制转IEEE 754：

1. 确定符号位。
2. 转换为二进制。
3. 写成 $1.xxx \times 2^e$。
4. 计算阶码 $E = e + Bias$。
5. 去掉隐藏的最高位1。
6. 补齐尾数字段，必要时舍入。
7. 拼接并转换为十六进制。

十三、IEEE 754特殊值 必会

特别注意：不是所有浮点数都使用 $1.F \times 2^{E-Bias}$ 公式。

阶码字段	尾数字段	表示
全0	全0	正零或负零
全0	非0	非规格化数（次正规数）
非全0、非全1	任意	规格化有限数
全1	全0	正无穷或负无穷
全1	非0	NaN

1. 非规格化数

单精度：

$$\boxed{N = (-1)^S \times (0.F)_2 \times 2^{-126}}$$

注意：

- 隐含整数位是0。
- 指数不是 -127，而是 -126。

2. 无穷大

阶码全1，尾数字段全0。

例如：

$$+\infty$$

对应符号位0。

3. NaN

Not a Number，表示非数值结果。

例如浮点运算：

$$0.0/0.0$$

通常产生NaN。

4. 重要数值范围

IEEE 754单精度：

最小正规格化数：

$$\boxed{2^{-126}}$$

最小正非规格化数：

$$\boxed{2^{-149}}$$

最大有限正数：

$$\boxed{(2-2^{-23})\times 2^{127}}$$

十四、浮点数加减法 必会

浮点加减法的固定流程：

1. 对阶
↓
2. 尾数加减
↓
3. 结果规格化
↓
4. 舍入处理
↓
5. 判断上溢、下溢及特殊情况

1. 对阶

两浮点数相加，首先要使阶码一致。

通常：

小阶向大阶看齐。

例如：

$$A=1.101\times 2^4$$

$$B=1.110\times 2^2$$

要把B的阶码调整为4。

阶码增加2，尾数右移2位：

$$B=0.01110\times 2^4$$

于是：

$$A=1.10100\times 2^4$$

$$B = 0.01110 \times 2^4$$

2. 尾数相加

$$1.10100 + 0.01110$$

得到：

$$10.00010$$

所以：

$$A + B = 10.00010 \times 2^4$$

3. 规格化

$$10.00010 \times 2^4$$

小数点左移一位，阶码加1：

$$\boxed{1.000010 \times 2^5}$$

4. 舍入

如果尾数超过能够保存的位数，需要舍入。

IEEE 754默认舍入模式通常为：

就近舍入，遇到正好居中时取偶数（roundTiesToEven）。

5. 判断溢出

规格化、舍入后检查指数范围。

- 指数过大：可能上溢。
- 指数太小：可能需要使用非规格化数，或进一步下溢。

注意：阶码低于最小规格化指数，不代表一定直接变为0。

必背口诀

> 小阶对大阶，尾数做加减；结果要规格化，最后舍入查溢出。

十五、浮点数乘除法 重点

1. 浮点乘法

两个浮点数：

$$A = (-1)^{S_1} M_1 2^{e_1}$$

$$B = (-1)^{S_2} M_2 2^{e_2}$$

乘积：

$$\boxed{AB = (-1)^{S_1 \oplus S_2} (M_1 M_2) 2^{e_1 + e_2}}$$

固定步骤：

1. 符号异或。
2. 尾数相乘。
3. 真实阶码相加。
4. 规格化。
5. 舍入。
6. 检查异常和溢出。

如果直接使用偏置阶码字段：

$$\boxed{E = E_1 + E_2 - Bias}$$

这个公式针对正常有限数的阶码初步计算，后续规格化仍可能修改阶码。

2. 浮点除法

$$\boxed{\frac{A}{B} = (-1)^{S_1 \oplus S_2} \frac{M_1}{M_2} 2^{e_1 - e_2}}$$

若使用偏置阶码：

$$\boxed{E = E_1 - E_2 + Bias}$$

仍需进一步规格化、舍入并处理特殊值。

十六、浮点数精度与误差 重点

1. 为什么浮点数存在误差?

因为二进制浮点数只能精确表示有限精度内的某些数。

例如：

$$0.1_{10}$$

无法表示为有限位二进制小数。

因此浮点数通常只能存储其近似值。

2. 浮点数精度

单精度有效二进制位数：

$$\boxed{24\text{ 位}}$$

双精度：

$$\boxed{53\text{ 位}}$$

这里包括规格化数隐藏的最高位1。

对应十进制有效数字大致为：

- 单精度：约7位。
- 双精度：约15～16位。

3. ULP

ULP可理解为给定数值附近相邻可表示浮点数之间的间距。

例如单精度规格化数在：

$$[1,2)$$

范围内，间距为：

$$\boxed{2^{-23}}$$

在：

$$[2,4)$$

范围内，间距为：

$$\boxed{2^{-22}}$$

浮点数的绝对间距通常随数量级增大而增大。

十七、ALU与加法器 重点

1. ALU

算术逻辑单元，Arithmetic Logic Unit。

主要完成：

- 加法
- 减法
- 与、或、异或等逻辑运算
- 比较
- 移位相关操作（具体可能由独立移位器完成）

2. 一位全加器

输入：

- A
- B
- 低位进位 C_{in}

输出：

- 和 S
- 进位 C_{out}

求和：

$$\boxed{S = A \oplus B \oplus C_{in}}$$

进位：

$$\boxed{C_{out} = AB + (A \oplus B)C_{in}}$$

3. 串行进位加法器

也称行波进位加法器。

特点：

- 结构简单。
- 高位进位需要等待低位进位。

- 位数越多，进位传播延迟可能越大。

4. 并行进位加法器

提前计算进位，减少等待。

定义：

$$G_i = A_i B_i$$

$$P_i = A_i \oplus B_i$$

那么：

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

其中：

- G_i：进位产生信号。
- P_i：进位传递信号。

展开：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

这样可以通过组合逻辑提前计算多个进位。

5. 加法器性能比较

类型	优点	缺点
行波进位加法器	简单、面积较小	进位传播较慢
先行进位加法器	运算速度较快	电路较复杂
多级先行进位加法器	适合较宽字长	设计更加复杂

十八、408常见题型与固定步骤

这一部分是最值得直接背的内容。

题型1：原码、反码、补码转换

题目特征：

- 给定整数真值。
- 给定机器字长。
- 求原码、反码、补码。

固定步骤：

1. 将绝对值转换为二进制。
2. 补齐数值位。
3. 根据正负设置符号位。
4. 负数反码：数值位取反。
5. 负数补码：反码加1。

特殊注意：

- 最小负数不能使用同位数原码表示。
- 补码负数的符号位参与编码，不是独立数值符号。

题型2：补码转真值

看到补码：

1. 判断最高位。
2. 最高位0：直接转十进制。
3. 最高位1：将整个编码当无符号数转换，再减 2^n。

$$\boxed{x = U - 2^n}$$

其中U为补码位模式的无符号解释值。

题型3：判断溢出

看到补码加减法：

1. 将减法变为加法。
2. 完成n位运算。
3. 判断操作数及结果符号。
4. 或用符号位进位异或：

$$\boxed{OF = C_{in} \oplus C_{out}}$$

题型4：移位运算

1. 先确定有符号还是无符号。
2. 确定逻辑移位还是算术移位。
3. 左移低位补0。
4. 逻辑右移高位补0。
5. 补码算术右移高位补符号位。
6. 最后检查位数截断与溢出。

题型5：IEEE 754十进制转浮点

1. 确定符号。
2. 十进制转二进制。
3. 规格化。
4. 计算偏置阶码。
5. 去隐藏位。
6. 补齐或舍入尾数。
7. 拼接编码。

题型6：IEEE 754浮点转十进制

1. 拆出S、E、F。
2. 检查是否特殊值。
3. 计算真实阶码。
4. 恢复有效数字。
5. 代入公式：

$$\boxed{N = (-1)^S (1.F)_2 2^{E-Bias}}$$

该公式只适用于规格化有限数。

题型7：浮点数加减法

直接记：

$$\boxed{\text{对阶} \rightarrow \text{尾数运算} \rightarrow \text{规格化} \rightarrow \text{舍入} \rightarrow \text{异常检查}}$$

题型8：浮点数乘除法

乘法：

$$e = e_1 + e_2$$

除法：

$$e = e_1 - e_2$$

符号：

$$S = S_1 \oplus S_2$$

最后进行规格化、舍入及异常处理。

十九、考前公式速查表

知识点	公式
n位无符号整数范围	$0 \sim 2^n - 1$
n位原码整数范围	$-(2^{n-1} - 1) \sim 2^{n-1} - 1$
n位补码整数范围	$-2^{n-1} \sim 2^{n-1} - 1$
负数补码	$2^n - \lvert x \rvert$
常规移码	$x + 2^{n-1}$
补码负数求真值	$U - 2^n$
补码加法溢出	$C_{in} \oplus C_{out}$
单精度Bias	127
双精度Bias	1023
单精度有效位数	24
双精度有效位数	53
浮点乘法阶码	$e_1 + e_2$
浮点除法阶码	$e_1 - e_2$
浮点乘除法符号	$S_1 \oplus S_2$
全加器求和	$A \oplus B \oplus C_{in}$
全加器进位	$AB + (A \oplus B)C_{in}$

二十、易错点总结

常见错误	正确理解
8位补码10000000表示负零	表示-128
补码范围与原码相同	补码多表示一个最小负数
CF=OF	两者含义不同
负数逻辑右移要补1	逻辑右移补0
补码算术右移补0	补符号位
相同二进制编码代表相同数值	还需看有符号或无符号
IEEE 754阶码直接等于真实指数	要减偏置值
单精度尾数字段23位，所以精度23位	规格化数包含隐藏位，共24位
IEEE 754阶码全0仍按隐藏位1处理	非规格化数隐藏整数位是0
阶码全1表示无穷大	尾数字段非0时表示NaN
浮点加法先加尾数再对阶	必须先对阶
浮点数的小数都可以精确表示	部分十进制小数只能近似存储

二十一、基础自测：8道题

数据的表示和运算：基础练习 0/8 已答

1. 8位补码11111011对应的十进制真值是多少？

A. -5

B. -4

C. 251

D. -123

2. 8位补码表示范围是什么？

A. -127～127

B. -128～127

C. -128～128

D. 0～255

3. 8位补码11110000算术右移1位后是什么？

A. 01111000

B. 11111000

C. 11110001

D. 01110000

4. 两个8位补码正数01111111与00000001相加，OF是多少？

A. 0

B. 1

C. 无法判断

D. 取决于CF

5. IEEE 754单精度的阶码偏置值是什么？

A. 128

B. 127

C. 1023

D. 255

6. 单精度规格化数的有效二进制精度是多少位?

A. 23

B. 24

C. 32

D. 31

7. 浮点数加减法的正确基本顺序是?

A. 加减→对阶→舍入

B. 对阶→尾数运算→规格化→舍入

C. 规格化→对阶→加减

D. 舍入→对阶→加减

8. 补码一位乘法Booth算法中，Q0Q-1=10时通常如何处理?

A. 加被乘数

B. 减被乘数

C. 不加不减

D. 直接结束

二十二、针对你目前复习情况的拿分顺序

这一章的知识点比较多，但不必全部投入同等精力。

优先顺序	复习内容	要达到的水平
1	原码、反码、补码、移码	看到编码立即转换
2	补码加减与溢出	会直接计算CF、OF
3	移位、符号扩展、类型转换	不在基础选择题丢分
4	IEEE 754编码	会十进制与十六进制相互转换
5	浮点数加减运算	记住对阶、规格化、舍入步骤
6	全加器、先行进位	掌握基本公式
7	定点乘除算法	理解原理并练典型真题

这一章最值得反复练习的是四类题:

- **补码转换题**：按固定字长转换正负整数。
- **溢出判断题**：看符号变化或计算进位异或。
- **IEEE 754编码题**：按符号、阶码、尾数依次处理。
- **浮点加减题**：先对阶，再运算、规格化、舍入。

对于你目前的目标，建议先把这四类基础题练到能够独立解答，再逐步增加Booth乘法、除法和进位电路分析的练习量。这样比一开始平均分配时间更容易形成稳定的得分能力。